

PIFF2502.0054

2.0

Gerado por Doxygen 1.9.8

1 PIFF-0054-ng	1
1.1 1. Arquitetura	1
1.1.1 1.1 Diagrama de Contexto (C4 Nível 1)	1
1.1.2 1.2 Diagrama de Containers (C4 Nível 2)	1
1.1.3 1.3 Diagrama de Sequência — Fluxo Principal	2
1.1.4 1.4 Diagrama de Componentes (C4 Nível 3) — <tt>src/</tt>	2
1.2 2. Pré-requisitos	3
1.3 3. Instalação	3
1.3.1 Dependências (<tt>requirements.txt</tt>)	3
1.4 4. Configuração	3
1.4.1 4.1 Variáveis de Conexão SQL Server	3
1.4.2 4.2 Variáveis de Cache	3
1.4.3 4.3 Exemplo <tt>.env</tt>	4
1.5 </blockquote>	4
1.6 5. Uso	4
1.6.1 5.1 Executar o Painel	4
1.6.2 5.2 Fluxo de Operação	4
1.6.3 5.3 Funcionalidades da Interface	4
1.7 6. Testes	4
1.8 7. Performance SQL	5
1.8.1 Estratégia de índices	5
1.9 8. Estrutura do Projeto	5
1.10 9. Licença	6
2 Engenharia Reversa do Banco SQL Server Projeto_54	7
2.1 1) Escopo e metodo	7
2.2 2) Visao geral do modelo (escopo 01 a 06)	7
2.2.1 2.1 Tabelas em escopo	7
2.2.2 2.2 Tabelas fora de escopo	8
2.2.3 2.3 Volume de dados no escopo	8
2.3 3) Padroes de schema no escopo	8
2.3.1 3.1 Padrao dinamico (exemplo: Cilindro_01)	8
2.3.2 3.2 Padrao estatico (exemplo: Cilindro_01_Estatico)	8
2.4 4) Integridade referencial no escopo	8
2.5 5) Indices observados no escopo	8
2.5.1 5.1 Tabelas dinamicas (01 a 06)	8
2.5.2 5.2 Tabelas estaticas (01 a 06)	9
2.6 6) Como a aplicacao consulta o banco (impacto em performance)	9
2.7 7) Melhorias de desempenho recomendadas (priorizadas, escopo 01 a 06)	9
2.8 P0 (alta prioridade)	9
2.9 P1 (alta prioridade)	9
2.10 P2 (media prioridade)	9

2.11 P3 (evolutivo)	9
2.12 8) Plano de implementacao sugerido para outro agente de IA	10
2.13 9) SQL util para validacao rapida (somente 01 a 06)	10
2.14 10) Conclusao	10
3 Relatório de Performance — PIFF-0054 · Consulta C01_TESTE_001	11
3.1 </blockquote>	11
3.2 1. Pipeline de Carga (Sequência Temporal)	11
3.3 2. Distribuição de Tempo por Subsistema	11
3.4 3. Fluxograma do Problema	12
3.5 4. Detalhamento por Etapa	12
3.5.1 4.1 Metadados de Colunas (<tt>_get_columns</tt>)	12
3.5.2 4.2 Descoberta de Tabela Estática (<tt>_find_static_table</tt>)	12
3.5.3 4.3 Query Dinâmica — O Gargalo	12
3.5.3.1 Por que 12 segundos para 100 linhas?	12
3.5.4 4.4 Query Estática	13
3.6 </blockquote>	13
3.7 5. Causa Raiz	13
3.8 6. Solução	13
3.8.1 Índice Ausente (já documentado no projeto)	13
3.8.2 Impacto Estimado	13
3.9 7. Recomendações	14
3.10 8. Notas Técnicas	14
4 Relatório Final: Projeto_54 — Comparativo de 5 Estratégias	15
4.1 1. As 5 Opções em Competição	15
4.2 2. Resultados dos Benchmarks	15
4.2.1 Consulta de Um Único Ensaio (Cilindro 01, ensaio 001 — 100 linhas)	15
4.2.2 Todos os Dados de Um Cilindro (2.500 linhas)	15
4.2.3 25 Consultas Sequenciais (lote — aplicação real)	16
4.2.4 Consulta Cross-Cilindro	16
4.3 3. Análise	16
4.3.1 Comparação Direta: Old + Redis vs HP + Redis	16
4.3.2 Por que Old SQL é mais rápido que HP SQL (sem cache)?	16
4.3.3 Comparação Visual Completa	16
4.4 4. Implementação do Redis	17
4.4.1 Arquitetura Atual	17
4.4.2 Fluxo de Funcionamento	17
4.4.3 Fallback Automático	17
4.5 5. Conclusão	17
4.5.1 Tabela Decisão Final	17
4.5.2 Veredito Final	17
4.5.3 Comandos de Operação	18

5 Namespaces	19
5.1 Lista de pacotes	19
6 Índice Hierárquico	21
6.1 Hierarquia de Classes	21
7 Índice dos Componentes	23
7.1 Lista de Classes	23
8 Índice dos Arquivos	25
8.1 Lista de Arquivos	25
9 Namespace	27
9.1 Referência do Namespace src	27
9.1.1 Descrição detalhada	27
9.2 Referência do Namespace src.application	27
9.2.1 Descrição detalhada	27
9.3 Referência do Namespace src.application.dto	27
9.3.1 Descrição detalhada	27
9.4 Referência do Namespace src.application.services	28
9.4.1 Descrição detalhada	28
9.5 Referência do Namespace src.application.validators	28
9.5.1 Descrição detalhada	28
9.5.2 Funções	28
9.5.2.1 validate_cylinder()	28
9.5.2.2 validate_obs_text()	28
9.5.2.3 validate_observation_command()	29
9.5.2.4 validate_observation_lookup()	29
9.5.2.5 validate_test_id()	29
9.5.2.6 validate_y_column_name()	29
9.6 Referência do Namespace src.config	29
9.6.1 Descrição detalhada	29
9.7 Referência do Namespace src.config.settings	29
9.7.1 Descrição detalhada	30
9.7.2 Funções	30
9.7.2.1 _as_bool()	30
9.7.3 Variáveis	30
9.7.3.1 SETTINGS	30
9.8 Referência do Namespace src.domain	30
9.8.1 Descrição detalhada	30
9.9 Referência do Namespace src.domain.errors	30
9.9.1 Descrição detalhada	30
9.10 Referência do Namespace src.domain.models	31
9.10.1 Descrição detalhada	31

9.11 Referência do Namespace <code>src.domain.repositories</code>	31
9.11.1 Descrição detalhada	31
9.12 Referência do Namespace <code>src.infrastructure</code>	31
9.12.1 Descrição detalhada	31
9.13 Referência do Namespace <code>src.infrastructure.cache</code>	31
9.13.1 Descrição detalhada	32
9.14 Referência do Namespace <code>src.infrastructure.cache_redis</code>	32
9.14.1 Descrição detalhada	32
9.14.2 Funções	32
9.14.2.1 <code>_cache_decoder()</code>	32
9.14.3 Variáveis	32
9.14.3.1 <code>_HAS_REDIS</code>	32
9.14.3.2 <code>_redis</code>	32
9.15 Referência do Namespace <code>src.infrastructure.exporters</code>	32
9.15.1 Descrição detalhada	33
9.16 Referência do Namespace <code>src.infrastructure.exporters.csv_exporter</code>	33
9.16.1 Descrição detalhada	33
9.17 Referência do Namespace <code>src.infrastructure.exporters.pdf_exporter</code>	33
9.17.1 Descrição detalhada	33
9.18 Referência do Namespace <code>src.infrastructure.sql</code>	33
9.18.1 Descrição detalhada	33
9.19 Referência do Namespace <code>src.infrastructure.sql.connection</code>	33
9.19.1 Descrição detalhada	33
9.19.2 Funções	34
9.19.2.1 <code>_execute_on()</code>	34
9.20 Referência do Namespace <code>src.infrastructure.sql.repositories</code>	34
9.20.1 Descrição detalhada	34
9.21 Referência do Namespace <code>src.infrastructure.sql.safe_sql</code>	34
9.21.1 Descrição detalhada	34
9.21.2 Funções	34
9.21.2.1 <code>escape_identifier()</code>	34
9.21.2.2 <code>escape_table()</code>	35
9.21.3 Variáveis	35
9.21.3.1 <code>_FORBIDDEN</code>	35
9.22 Referência do Namespace <code>src.infrastructure.sql.schema</code>	35
9.22.1 Descrição detalhada	35
9.22.2 Funções	35
9.22.2.1 <code>cylinder_id_column()</code>	35
9.22.2.2 <code>dynamic_table_name()</code>	36
9.22.2.3 <code>static_table_candidates()</code>	36
9.22.3 Variáveis	36
9.22.3.1 <code>CANONICAL_OBSERVATION_COLUMNS</code>	36

9.22.3.2 DYNAMIC_DEFAULT_COLUMNS	36
9.22.3.3 DYNAMIC_EXCLUDE_COLUMNS	36
9.22.3.4 FORCE_COLUMN_CANDIDATES	36
9.23 Referência do Namespace src.ui	37
9.23.1 Descrição detalhada	37
9.24 Referência do Namespace src.ui.streamlit_app	37
9.24.1 Descrição detalhada	37
9.24.2 Funções	37
9.24.2.1 _build_services()	37
9.24.2.2 _friendly_error()	37
9.24.2.3 _plot_multiaxis()	37
9.24.2.4 main()	38
9.25 Referência do Namespace src.ui.theme	38
9.25.1 Descrição detalhada	38
9.25.2 Funções	38
9.25.2.1 inject_theme()	38
9.25.2.2 render_kpis()	38
9.25.2.3 render_signature_flow()	38
10 Classes	39
10.1 Referência da Classe src.infrastructure.cache_redis._CacheEncoder	39
10.1.1 Descrição detalhada	39
10.1.2 Documentação das funções	39
10.1.2.1 default()	39
10.2 Referência da Classe src.infrastructure.cache._Entry	39
10.2.1 Descrição detalhada	39
10.2.2 Atributos	40
10.2.2.1 expires_at	40
10.2.2.2 value	40
10.3 Referência da Classe src.domain.errors.AmbiguousMatchError	40
10.3.1 Descrição detalhada	40
10.4 Referência da Classe src.domain.repositories.CsvExporter	40
10.4.1 Descrição detalhada	40
10.4.2 Documentação das funções	41
10.4.2.1 export()	41
10.5 Referência da Classe src.domain.models.DiagnosticsInfo	41
10.5.1 Descrição detalhada	41
10.5.2 Atributos	41
10.5.2.1 active_context	41
10.5.2.2 checked_at	41
10.5.2.3 connection_message	41
10.5.2.4 connection_ok	41

10.5.2.5	odbc_drivers	41
10.6	Referência da Classe <code>src.domain.repositories.DiagnosticsRepository</code>	42
10.6.1	Descrição detalhada	42
10.6.2	Documentação das funções	42
10.6.2.1	diagnostics()	42
10.7	Referência da Classe <code>src.application.services.DiagnosticsService</code>	42
10.7.1	Descrição detalhada	42
10.7.2	Construtores e Destrutores	42
10.7.2.1	__init__()	42
10.7.3	Documentação das funções	43
10.7.3.1	get()	43
10.7.4	Atributos	43
10.7.4.1	repository	43
10.8	Referência da Classe <code>src.domain.errors.DomainError</code>	43
10.8.1	Descrição detalhada	43
10.9	Referência da Classe <code>src.domain.models.DynamicDataResult</code>	43
10.9.1	Descrição detalhada	43
10.9.2	Documentação das funções	44
10.9.2.1	total_rows()	44
10.9.3	Atributos	44
10.9.3.1	ordered_by	44
10.9.3.2	rows [1/2]	44
10.9.3.3	rows [2/2]	44
10.10	Referência da Classe <code>src.application.dto.ExportPdfRequest</code>	44
10.10.1	Descrição detalhada	44
10.10.2	Atributos	44
10.10.2.1	export_mode	44
10.10.2.2	graph_png	44
10.10.2.3	metadata	44
10.10.2.4	table_sample_max_rows	44
10.11	Referência da Classe <code>src.application.services.ExportService</code>	45
10.11.1	Descrição detalhada	45
10.11.2	Construtores e Destrutores	45
10.11.2.1	__init__()	45
10.11.3	Documentação das funções	45
10.11.3.1	export_csv()	45
10.11.3.2	export_pdf()	45
10.11.4	Atributos	46
10.11.4.1	csv_exporter	46
10.11.4.2	pdf_exporter	46
10.12	Referência da Classe <code>src.domain.errors.InfrastructureError</code>	46
10.12.1	Descrição detalhada	46

10.13 Referência da Classe <code>src.application.dto.LoadTestResponse</code>	46
10.13.1 Descrição detalhada	46
10.13.2 Atributos	46
10.13.2.1 <code>available_y_columns</code>	46
10.13.2.2 <code>force_column</code>	47
10.13.2.3 <code>meta</code>	47
10.13.2.4 <code>rows</code>	47
10.13.2.5 <code>static_row</code>	47
10.14 Referência da Classe <code>src.domain.errors.NotFoundError</code>	47
10.14.1 Descrição detalhada	47
10.15 Referência da Classe <code>src.domain.models.ObservationCommand</code>	47
10.15.1 Descrição detalhada	47
10.15.2 Atributos	48
10.15.2.1 <code>cyl_num</code>	48
10.15.2.2 <code>local_col</code>	48
10.15.2.3 <code>obs_text</code>	48
10.15.2.4 <code>test_id</code>	48
10.15.2.5 <code>y_column_name</code>	48
10.15.2.6 <code>y_value</code>	48
10.16 Referência da Classe <code>src.domain.repositories.ObservationCommandRepository</code>	48
10.16.1 Descrição detalhada	48
10.16.2 Documentação das funções	49
10.16.2.1 <code>delete_observation()</code>	49
10.16.2.2 <code>get_observation()</code>	49
10.16.2.3 <code>upsert_observation()</code>	49
10.17 Referência da Classe <code>src.domain.models.ObservationLookup</code>	49
10.17.1 Descrição detalhada	49
10.17.2 Atributos	49
10.17.2.1 <code>cyl_num</code>	49
10.17.2.2 <code>local_col</code>	49
10.17.2.3 <code>test_id</code>	50
10.17.2.4 <code>y_column_name</code>	50
10.17.2.5 <code>y_value</code>	50
10.18 Referência da Classe <code>src.domain.models.ObservationRecord</code>	50
10.18.1 Descrição detalhada	50
10.18.2 Atributos	50
10.18.2.1 <code>obs_text</code>	50
10.18.2.2 <code>val_obs</code>	50
10.18.2.3 <code>y_column_name</code>	50
10.19 Referência da Classe <code>src.domain.models.ObservationResult</code>	50
10.19.1 Descrição detalhada	50
10.19.2 Atributos	51

10.19.2.1 columns_used	51
10.19.2.2 matched_rows	51
10.19.2.3 updated	51
10.20 Referência da Classe src.application.services.ObservationService	51
10.20.1 Descrição detalhada	51
10.20.2 Construtores e Destrutores	51
10.20.2.1 __init__()	51
10.20.3 Documentação das funções	51
10.20.3.1 delete()	51
10.20.3.2 get()	52
10.20.3.3 upsert()	52
10.20.4 Atributos	52
10.20.4.1 repository	52
10.21 Referência da Classe src.domain.repositories.PdfExporter	52
10.21.1 Descrição detalhada	52
10.21.2 Documentação das funções	52
10.21.2.1 export()	52
10.22 Referência da Classe src.domain.errors.PersistenceError	53
10.22.1 Descrição detalhada	53
10.23 Referência da Classe src.infrastructure.cache_redis.RedisCache	53
10.23.1 Descrição detalhada	53
10.23.2 Construtores e Destrutores	54
10.23.2.1 __init__()	54
10.23.3 Documentação das funções	54
10.23.3.1 __repr__()	54
10.23.3.2 _fallback_get()	54
10.23.3.3 _fallback_set()	54
10.23.3.4 available()	54
10.23.3.5 delete()	54
10.23.3.6 flush()	54
10.23.3.7 get()	55
10.23.3.8 set()	55
10.23.4 Atributos	55
10.23.4.1 _client	55
10.23.4.2 _connected	55
10.23.4.3 ttl	55
10.24 Referência da Classe src.infrastructure.exporters.csv_exporter.SemicolonCsvExporter	55
10.24.1 Descrição detalhada	55
10.24.2 Documentação das funções	55
10.24.2.1 export()	55
10.25 Referência da Classe src.config.settings.Settings	56
10.25.1 Descrição detalhada	56

10.25.2 Documentação das funções	56
10.25.2.1 <code>allowed_cylinders()</code>	56
10.25.3 Atributos	56
10.25.3.1 <code>cache_ttl_seconds</code>	56
10.25.3.2 <code>sql_database</code>	56
10.25.3.3 <code>sql_driver</code>	57
10.25.3.4 <code>sql_password</code>	57
10.25.3.5 <code>sql_server</code>	57
10.25.3.6 <code>sql_username</code>	57
10.25.3.7 <code>trusted_connection</code>	57
10.26 Referência da Classe <code>src.infrastructure.sql.repositories.SqlDiagnosticsRepository</code>	57
10.26.1 Descrição detalhada	57
10.26.2 Construtores e Destrutores	57
10.26.2.1 <code>__init__()</code>	57
10.26.3 Documentação das funções	58
10.26.3.1 <code>diagnostics()</code>	58
10.26.4 Atributos	58
10.26.4.1 <code>conn</code>	58
10.27 Referência da Classe <code>src.infrastructure.sql.repositories.SqlObservationRepository</code>	58
10.27.1 Descrição detalhada	58
10.27.2 Construtores e Destrutores	58
10.27.2.1 <code>__init__()</code>	58
10.27.3 Documentação das funções	59
10.27.3.1 <code>_match_query()</code>	59
10.27.3.2 <code>delete_observation()</code>	59
10.27.3.3 <code>get_observation()</code>	59
10.27.3.4 <code>upsert_observation()</code>	59
10.27.4 Atributos	60
10.27.4.1 <code>conn</code>	60
10.28 Referência da Classe <code>src.infrastructure.sql.connection.SqlServerConnection</code>	60
10.28.1 Descrição detalhada	60
10.28.2 Construtores e Destrutores	60
10.28.2.1 <code>__init__()</code>	60
10.28.3 Documentação das funções	60
10.28.3.1 <code>build_connection_string()</code>	60
10.28.3.2 <code>connect()</code>	61
10.28.3.3 <code>connect_shared()</code>	61
10.28.3.4 <code>execute()</code>	61
10.28.3.5 <code>execute_transaction()</code>	61
10.28.3.6 <code>fetch_all()</code>	61
10.28.3.7 <code>fetch_all_on()</code>	62
10.28.3.8 <code>list_drivers()</code>	62

10.28.4 Atributos	62
10.28.4.1 settings	62
10.29 Referência da Classe <code>src.infrastructure.sql.repositories.SqlTestQueryRepository</code>	62
10.29.1 Descrição detalhada	63
10.29.2 Construtores e Destrutores	63
10.29.2.1 <code>__init__()</code>	63
10.29.3 Documentação das funções	63
10.29.3.1 <code>_fetch()</code>	63
10.29.3.2 <code>_find_static_table()</code>	63
10.29.3.3 <code>_get_columns()</code>	63
10.29.3.4 <code>_table_exists()</code>	64
10.29.3.5 <code>detect_force_column()</code>	64
10.29.3.6 <code>list_test_ids()</code>	64
10.29.3.7 <code>load_available_y_columns()</code>	64
10.29.3.8 <code>load_dynamic_rows()</code>	64
10.29.3.9 <code>load_static_row()</code>	65
10.29.4 Atributos	65
10.29.4.1 <code>conn</code>	65
10.30 Referência da Classe <code>src.domain.models.StaticDataResult</code>	65
10.30.1 Descrição detalhada	65
10.30.2 Atributos	65
10.30.2.1 <code>row</code>	65
10.30.2.2 <code>table_used</code>	65
10.31 Referência da Classe <code>src.infrastructure.exporters.pdf_exporter.TechnicalPdfExporter</code>	66
10.31.1 Descrição detalhada	66
10.31.2 Documentação das funções	66
10.31.2.1 <code>export()</code>	66
10.32 Referência da Classe <code>src.domain.models.TestContext</code>	66
10.32.1 Descrição detalhada	66
10.32.2 Atributos	66
10.32.2.1 <code>cyl_num</code>	66
10.32.2.2 <code>test_id</code>	67
10.33 Referência da Classe <code>src.domain.repositories.TestQueryRepository</code>	67
10.33.1 Descrição detalhada	67
10.33.2 Documentação das funções	67
10.33.2.1 <code>detect_force_column()</code>	67
10.33.2.2 <code>list_test_ids()</code>	67
10.33.2.3 <code>load_available_y_columns()</code>	67
10.33.2.4 <code>load_dynamic_rows()</code>	68
10.33.2.5 <code>load_static_row()</code>	68
10.34 Referência da Classe <code>src.application.services.TestReadService</code>	68
10.34.1 Descrição detalhada	68

10.34.2 Construtores e Destrutores	68
10.34.2.1 <code>__init__()</code>	68
10.34.3 Documentação das funções	69
10.34.3.1 <code>list_test_ids()</code>	69
10.34.3.2 <code>load_test()</code>	69
10.34.4 Atributos	69
10.34.4.1 <code>__test__</code>	69
10.34.4.2 <code>cache</code>	69
10.34.4.3 <code>repository</code>	69
10.35 Referência da Classe <code>src.infrastructure.cache.TTLCache</code>	69
10.35.1 Descrição detalhada	70
10.35.2 Construtores e Destrutores	70
10.35.2.1 <code>__init__()</code>	70
10.35.3 Documentação das funções	70
10.35.3.1 <code>get()</code>	70
10.35.3.2 <code>set()</code>	70
10.35.4 Atributos	70
10.35.4.1 <code>ttl_seconds</code>	70
10.36 Referência da Classe <code>src.domain.errors.ValidationError</code>	70
10.36.1 Descrição detalhada	71
11 Arquivos	73
11.1 Referência do Arquivo <code>ENGENHARIA_REVERSA_SQL_PROJETO_54.md</code>	73
11.2 Referência do Arquivo <code>PERFORMANCE_REPORT_C01_TESTE_001.md</code>	73
11.3 Referência do Arquivo <code>PERFORMANCE_REPORT_PROJETO54_HP_FINAL.md</code>	73
11.4 Referência do Arquivo <code>README.md</code>	73
11.5 Referência do Arquivo <code>dto.py</code>	73
11.6 Referência do Arquivo <code>services.py</code>	73
11.7 Referência do Arquivo <code>validators.py</code>	74
11.8 Referência do Arquivo <code>settings.py</code>	74
11.9 Referência do Arquivo <code>errors.py</code>	74
11.10 Referência do Arquivo <code>models.py</code>	75
11.11 Referência do Arquivo <code>cache.py</code>	75
11.12 Referência do Arquivo <code>cache_redis.py</code>	75
11.13 Referência do Arquivo <code>csv_exporter.py</code>	75
11.14 Referência do Arquivo <code>pdf_exporter.py</code>	76
11.15 Referência do Arquivo <code>__init__.py</code>	76
11.16 Referência do Arquivo <code>application/__init__.py</code>	76
11.17 Referência do Arquivo <code>config/__init__.py</code>	76
11.18 Referência do Arquivo <code>domain/__init__.py</code>	76
11.19 Referência do Arquivo <code>infrastructure/__init__.py</code>	76
11.20 Referência do Arquivo <code>infrastructure/exporters/__init__.py</code>	77

11.21 Referência do Arquivo <code>infrastructure/sql/__init__.py</code>	77
11.22 Referência do Arquivo <code>ui/__init__.py</code>	77
11.23 Referência do Arquivo <code>connection.py</code>	77
11.24 Referência do Arquivo <code>domain/repositories.py</code>	77
11.25 Referência do Arquivo <code>infrastructure/sql/repositories.py</code>	78
11.26 Referência do Arquivo <code>safe_sql.py</code>	78
11.27 Referência do Arquivo <code>schema.py</code>	78
11.28 Referência do Arquivo <code>streamlit_app.py</code>	79
11.29 Referência do Arquivo <code>theme.py</code>	79

Capítulo 1

PIFF-0054-ng

Sistema de visualização técnica de ensaios industriais — persistência de observações por coordenada, exportação CSV/PDF e diagnósticos operacionais.

"" "[Licence](https://img.shields.io/badge/licence-Projeto_54-6bbf6b)"

1.1 1. Arquitetura

O sistema segue uma arquitetura em camadas (*clean architecture* simplificada) com separação explícita entre domínio, aplicação, infraestrutura e interface.

1.1.1 1.1 Diagrama de Contexto (C4 Nível 1)

```
C4Context
title Diagrama de Contexto -- PIFF-0054-ng

Person(eng, "Engenheiro de Ensaios", "Opera o painel técnico para visualizar ensaios, anotar observações e exportar relatórios")

System_Boundary(sistema, "PIFF-0054-ng") {
    System(painel, "Painel Técnico Streamlit", "Interface web para visualização, anotação e exportação de ensaios industriais")
}

System_Ext(sql, "SQL Server", "Banco Projeto_54\nTabelas dinâmicas e estáticas por cilindro")
System_Ext(redis, "Redis", "Cache opcional de consultas (fallback em memória)")

Rel(eng, painel, "Seleciona cilindro/teste, visualiza gráficos, anota observações, exporta CSV/PDF")
Rel(painel, sql, "Consulta dados dinâmicos/estáticos, CRUD de observações", "pyodbc")
Rel(painel, redis, "Cache de consultas frequentes (TTL 300s)", "redis-py")

UpdateLayoutConfig($c4ShapeInRow="3", $c4BoundaryInRow="1")
```

1.1.2 1.2 Diagrama de Containers (C4 Nível 2)

```
C4Container
title Diagrama de Containers -- Camadas internas

Person(eng, "Engenheiro", "Usuário do painel técnico")

System_Boundary(painel, "Painel Técnico") {
    Container(ui, "UI Streamlit", "Python · Streamlit", "Interface com gráficos Plotly multi-eixo, formulários de observação e exportação")
    Container(svc, "Application Services", "Python", "Casos de uso: TestReadService, ObservationService, ExportService, DiagnosticsService")
    Container(domain, "Domain", "Python", "Entidades, protocolos (repositórios) e erros de domínio")
    ContainerDb(infra, "Infrastructure", "Python", "SQL Server connection, Redis cache, exportadores CSV/PDF, SQL seguro")
}

System_Ext(sql, "SQL Server", "Projeto_54")
System_Ext(redis_ext, "Redis", "Cache opcional")

Rel(eng, ui, "Interage com", "Browser")
```

```

Rel(ui, svc, "Chama casos de uso")
Rel(svc, domain, "Usa entidades e protocolos")
Rel(svc, infra, "Implementa protocolos")
Rel(infra, sql, "Consulta", "pyodbc · T-SQL")
Rel(infra, redis_ext, "Cacheia", "redis-py · JSON")

UpdateLayoutConfig($c4ShapeInRow="3", $c4BoundaryInRow="1")

```

1.1.3 1.3 Diagrama de Sequência — Fluxo Principal

```

sequenceDiagram
    actor Eng as Engenheiro
    participant UI as Streamlit UI
    participant Svc as TestReadService
    participant Repo as SqlTestQueryRepository
    participant Cache as RedisCache
    participant DB as SQL Server

    Eng->>UI: Seleciona cilindro e ID_Teste
    UI->>Svc: load_test(cyl_num, test_id)
    Svc->>Cache: get("test:{cyl_num}:{test_id}")
    alt Cache hit
        Cache->>Svc: Dados em cache
    else Cache miss
        Svc->>Repo: load_dynamic_rows(ctx)
        Repo->>DB: SELECT colunas FROM Cilindro_XX WHERE ID_Teste = ?
        DB-->>Repo: Linhas dinâmicas
        Repo->>DB: SELECT * FROM Cilindro_XX_Estático WHERE ID_Teste = ?
        DB-->>Repo: Linha estática
        Repo-->>Svc: DynamicDataResult + StaticDataResult
        Svc->>Cache: set("test:...", response, TTL=300)
    end
    Svc-->>UI: LoadTestResponse
    UI-->>Eng: Gráfico multi-eixo + tabelas

    Eng->>UI: Anota observação numa coordenada
    UI->>Svc: upsert(ObservationCommand)
    Svc->>Repo: upsert_observation(cmd)
    Repo->>DB: UPDATE ... SET OBS=?, val_obs=?
    DB-->>Repo: matched_rows
    Repo-->>Svc: ObservationResult
    Svc-->>UI: Resultado
    UI-->>Eng: Confirmação

```

1.1.4 1.4 Diagrama de Componentes (C4 Nível 3) — <tt>src/</tt>

```

flowchart TD
    subgraph domain["domain"]
        models["models.py"]
        repositories["repositories.py"]
        errors["errors.py"]
    end

    subgraph application["application"]
        services["services.py"]
        validators["validators.py"]
        dto["dto.py"]
    end

    subgraph infrastructure["infrastructure"]
        sql["sql"]
        cache["cache"]
        exporters["exporters"]
    end

    subgraph ui["ui"]
        streamlit["streamlit_app.py"]
        theme["theme.py"]
    end

    services --> models
    services --> repositories
    services --> validators
    services --> dto
    sql --> repositories
    cache --> services
    exporters --> services
    streamlit --> services
    streamlit --> theme

    style domain fill:#1a1f24,stroke:#f0883e,color:#ece8de
    style application fill:#1a1f24,stroke:#5b9bd5,color:#ece8de
    style infrastructure fill:#1a1f24,stroke:#6bbf6b,color:#ece8de
    style ui fill:#1a1f24,stroke:#e8c84a,color:#ece8de

```

1.2 2. Pré-requisitos

Recurso	Versão / Observação
Python	3.11 ou superior
SQL Server	2019+ com banco Projeto_54 criado
Driver ODBC	ODBC Driver 18 for SQL Server (ou 17) — instalação Linux
Redis	Opcional — se indisponível, usa fallback em memória (TTLCache)

1.3 3. Instalação

```
# 1. Clone o repositório
git clone <url-do-repositorio>
cd PIFF0054-ng

# 2. Crie e ative o ambiente virtual
python -m venv .venv
source .venv/bin/activate           # Linux/macOS
# .venv\Scripts\Activate.ps1       # Windows PowerShell

# 3. Instale as dependências
pip install -r requirements.txt
```

1.3.1 Dependências (<tt>requirements.txt</tt>)

Pacote	Versão	Finalidade
streamlit	1.37.0	Interface web
pandas	2.2.2	Manipulação de dados
plotly	5.23.0	Gráficos multi-eixo interativos
pyodbc	5.1.0	Conexão SQL Server
reportlab	4.2.2	Geração de PDF técnico
redis	5.0.0	Cache Redis (fallback opcional)
pytest	8.3.2	Testes
pytest-cov	5.0.0	Cobertura de testes

1.4 4. Configuração

O sistema é configurado exclusivamente por **variáveis de ambiente**. Crie um arquivo `.env` na raiz do projeto ou exporte as variáveis diretamente no shell.

1.4.1 4.1 Variáveis de Conexão SQL Server

Variável	Padrão	Descrição
PIFF_SQL_SERVER	localhost	Host do SQL Server
PIFF_SQL_DATABASE	Projeto_54	Nome do banco de dados
PIFF_SQL_USERNAME	sa	Usuário (ignorado se TRUSTED↔_CONNECTION=1)
PIFF_SQL_PASSWORD	P@ssw0rd_Projeto54!	Senha (ignorado se TRUSTED↔_CONNECTION=1)
PIFF_SQL_DRIVER	ODBC Driver 18 for SQL Server	Nome do driver ODBC instalado
PIFF_SQL_TRUSTED_↔_CONNECTION	false	1 / true para autenticação Windows

1.4.2 4.2 Variáveis de Cache

Variável	Padrão	Descrição
PIFF_CACHE_TTL_SECONDS	300	Tempo de vida do cache (5 min)
PIFF_REDIS_HOST	localhost	Host Redis (fallback em memória se indisponível)
PIFF_REDIS_PORT	6379	Porta Redis

1.4.3 4.3 Exemplo `<tt>.env</tt>`

```
PIFF_SQL_SERVER=192.168.1.100
PIFF_SQL_DATABASE=Projeto_54
PIFF_SQL_USERNAME=sa
PIFF_SQL_PASSWORD=MinhaSenhaAqui
PIFF_SQL_DRIVER=ODBC Driver 18 for SQL Server
PIFF_CACHE_TTL_SECONDS=300
```

Cilindros permitidos: 1 a 6 (definido em `Settings.allowed_cylinders`).

1.5 `</blockquote>`

1.6 5. Uso

1.6.1 5.1 Executar o Painel

```
streamlit run app.py
```

O painel será aberto em <http://localhost:8501>.

1.6.2 5.2 Fluxo de Operação

```
flowchart LR
    A[Selecionar Cilindro<br/>1-6] --> B[Selecionar ID_Teste<br/>Lista dinâmica do banco]
    B --> C[Carregar Ensaio]
    C --> D{Ensaio carregado?}
    D -- Sim --> E[Visualizar Gráfico<br/>Multi-eixo Plotly]
    D -- Não --> A
    E --> F[Escolher Variáveis Y]
    F --> G[Anotar Observação<br/>por Coordenada]
    G --> H[Salvar / Excluir<br/>Observação]
    H --> I[Exportar CSV]
    I --> J[Exportar PDF Técnico]
    J --> K[Aba Diagnósticos<br/>Conexão e contexto]
```

1.6.3 5.3 Funcionalidades da Interface

Aba / Seção	Descrição
Sidebar	Seleção de cilindro (1–6) e ID_Teste com botão "Carregar ensaio"
Gráfico multi-eixo	Plotly interativo com eixos Y sobrepostos, cores da paleta industrial
Observação por coordenada	Seleciona LocalCol + variável Y → consulta observação existente → edita/salva/exclui
Aba "Operação"	Gráfico + formulário de observação
Aba "Evidências"	Exportação CSV (separador ;) e PDF técnico (com gráfico, dados estáticos e amostra dinâmica)
Aba "Diagnósticos"	Estado da conexão SQL Server, drivers ODBC instalados, contexto ativo (JSON)

1.7 6. Testes

```
# Executa todos os testes (unitários + integração)
pytest

# Com cobertura
pytest --cov=src --cov-report=term-missing

# Apenas testes unitários (sem banco)
pytest tests/unit/

# Apenas testes de integração (requer SQL Server acessível)
```

pytest tests/integration/
Os testes de UI utilizam Streamlit + Playwright para testes end-to-end com navegador headless.

1.8 7. Performance SQL

Scripts versionados em sql/performance/ — recorte de otimização (01–06):

Script	Descrição
01_baseline_benchmark.sql	Benchmark inicial das consultas
02_create_composite_indexes.sql	Cria índices compostos (ID_Testes, LocalCol) INCLUDE (colunas úteis) para eliminar <i>key lookups</i>
03_update_stats_and_index_review.sql	Atualiza estatísticas e revisa a eficácia dos índices

1.8.1 Estratégia de índices

- Índice composto por cilindro: IX_Cilindro_XX_ID_Testes_LocalCol
- INCLUDE com colunas do conjunto DYNAMIC_DEFAULT_COLUMNS → *covering index scan*
- Redução de ~12s (full table scan) para milissegundos (index seek)

1.9 8. Estrutura do Projeto

```
PIFF0054-ng/  
app.py                # Entrypoint: streamlit run app.py  
requirements.txt      # Dependências Python  
pytest.ini            # Configuração do pytest  
.env.example          # Template de configuração  
src/  
  __init__.py  
  
  domain/  
    models.py          # Núcleo do domínio  
    repositories.py    # Entidades (dataclasses frozen)  
    errors.py          # Protocolos (interfaces)  
    # Hierarquia de exceções  
  
  application/  
    services.py        # Casos de uso  
    validators.py      # TestReadService, ObservationService, etc.  
    dto.py             # Validações de domínio  
    # Data Transfer Objects  
  
  infrastructure/  
    sql/  
      connection.py    # Adaptadores externos  
      repositories.py  # SqlServerConnection (ODBC)  
      safe_sql.py      # Implementações SQL dos protocolos  
      schema.py        # Escape de identificadores (brackets)  
      cache.py         # Nomes canônicos de tabelas/colunas  
      cache_redis.py   # TTLCache em memória  
      exporters/  
        csv_exporter.py # RedisCache com fallback  
        pdf_exporter.py # SemicolonCsvExporter (separador ;)  
                        # TechnicalPdfExporter (ReportLab)  
  
  ui/  
    streamlit_app.py   # Interface com o usuário  
    theme.py          # Aplicação Streamlit principal  
    # Tema industrial "Steel & Amber"  
  
  tests/  
    unit/  
      test_safe_sql.py # Testes unitários (sem banco)  
      test_services.py  
      test_validators.py  
    integration/  
      test_db_queries.py # Testes de integração (requer SQL Server)  
      test_sql_repositories.py  
    ui/  
      test_streamlit_app.py # Testes de interface (Streamlit + Playwright)  
      test_streamlit_e2e.py  
  
  sql/  
    03_create_projeto_54_ng.sql # Schema do banco  
    04_create_projeto_54_hp.sql # Schema high-performance (v1-v3)  
    migrate_to_projeto_54_ng.py # Migração de dados
```

```
migrate_to_projeto_54_hp.py      # Migração HP
performance/                     # Scripts de otimização
    01_baseline_benchmark.sql
    02_create_composite_indexes.sql
    03_update_stats_and_index_review.sql
backups/                         # Backups do banco (.bak)
docs/                           # Documentação técnica adicional
    ENGENHARIA_REVERSA_SQL_PROJETO_54.md
    PERFORMANCE_REPORT_C01_TESTE_001.md
    PERFORMANCE_REPORT_PROJETO54_HP_FINAL.md
spec/                          # Especificações de projeto
    spec-architecture-refatoracao-report-piff54-ng.md
    spec-process-backlog-operacional-checklist-refatoracao-piff54-ng.md
...
imagens/                        # Recursos visuais
```

1.10 9. Licença

Proprietária — Projeto_54. Todos os direitos reservados.

Capítulo 2

Engenharia Reversa do Banco SQL Server Projeto_54

Data da análise: 2026-06-26

Origem: consultas diretas no SQL Server Express (localhost\SQLEXPRESS, banco Projeto_54) + leitura do código de acesso a dados em Report_PIFF54/modules/dal.py.

2.1 1) Escopo e metodo

Este documento foi gerado para uso por outro agente de IA.

Escopo considerado nesta versao:

- somente tabelas de cilindro 01 a 06 (dinamicas e estaticas)
- tabelas de 07 a 13 foram desconsideradas neste documento por inconsistencia no numero de colunas em relacao ao padrao adotado para 01 a 06

Objetivos:

- mapear estrutura real do recorte (01 a 06)
- identificar pontos de melhoria de desempenho dentro do recorte
- fornecer plano de implementacao focado no escopo valido

Metodologia aplicada:

- catalogo: sys.tables, INFORMATION_SCHEMA.COLUMNS
- volume: sys.partitions
- integridade: INFORMATION_SCHEMA.TABLE_CONSTRAINTS, sys.foreign_keys
- indices: sys.indexes
- comportamento da aplicacao: leitura do DAL (load_test_ids, load_all_test_data, load_static_record)

2.2 2) Visao geral do modelo (escopo 01 a 06)

2.2.1 2.1 Tabelas em escopo

- dinamicas: Cilindro_01, Cilindro_02, Cilindro_03, Cilindro_04, Cilindro_05, Cilindro_06
- estaticas: Cilindro_01_Estatico/Estatico, Cilindro_02_Estatico/Estatico, Cilindro_03_Estatico/Estatico, Cilindro_04_Estatico/Estatico, Cilindro_05_Estatico/Estatico, Cilindro_06_Estatico/Estatico

2.2.2 2.2 Tabelas fora de escopo

- Cilindro_07 a Cilindro_13 (dinamicas e estaticas)
- motivo: inconsistencias de quantidade de colunas em relacao ao modelo usado como referencia neste documento

2.2.3 2.3 Volume de dados no escopo

- dinamicas (01 a 06): tipicamente 2.500 linhas por cilindro
- estaticas (01 a 06): tipicamente 25 linhas por cilindro

2.3 3) Padroes de schema no escopo

2.3.1 3.1 Padrao dinamico (exemplo: Cilindro_01)

Colunas principais observadas:

- chave tecnica: RowId (bigint)
- chave de negocio: Cilindro_01_ID_Testes (nvarchar, NOT NULL)
- serie temporal: TimeCol, MSecCol, LocalCol
- sinais: Setpoint, Forca, Pressao_Compressor, Pressao_Reguladora, Temperatura_Ambiente
- estado: Tipo_Testes, LeituraIndice, Em_Alarme, CreatedAt
- observacoes por coordenada: OBS, val_obs, y_column_name

2.3.2 3.2 Padrao estatico (exemplo: Cilindro_01_Estatico)

Colunas principais observadas:

- chave de negocio: Cilindro_01_ID_Testes (nvarchar, NOT NULL)
- identificacao: Cilindro, Sensor_Cap, Tipo_Testes, Status_Ensaio
- parametros: Setpoint, Duracao_Estimada, Temperatura_Inicial
- tempos consolidados: Tempo_Inicial, Tempo_Final, Tempo_Decorrido_*
- auditoria: CreatedAt

2.4 4) Integridade referencial no escopo

- os pares de 01 a 06 apresentam PK e FK
- FK tipica: Cilindro_nn.Cilindro_nn_ID_Testes -> Cilindro_nn_Estatico.Cilindro_nn_ID_Testes

2.5 5) Indices observados no escopo

2.5.1 5.1 Tabelas dinamicas (01 a 06)

Padrao observado:

- PK clustered
- IX_Cilindro_nn_ID_Testes (nonclustered)
- IX_Cilindro_nn_LocalCol (nonclustered)

2.5.2 5.2 Tabelas estaticas (01 a 06)

Padrao observado:

- PK clustered
- indice nonclustered filtrado por Cilindro_nn_ID_Testes IS NOT NULL

2.6 6) Como a aplicacao consulta o banco (impacto em performance)

Padroes de query em modules/dal.py:

- lista de ensaios: `SELECT DISTINCT Cilindro_nn_ID_Testes ... ORDER BY Cilindro_nn_ID_Testes`
- carga principal: `SELECT <colunas> FROM Cilindro_nn WHERE Cilindro_nn_ID_Testes = ?`
- ordenacao por tempo no app (python) em parte do fluxo; em alguns casos ha `ORDER BY LocalCol`
- carga estatica: `SELECT * FROM Cilindro_nn_Estatico WHERE Cilindro_nn_ID_Testes = ?`
- leitura de metadados: `INFORMATION_SCHEMA.COLUMNS`

Consequencia no escopo 01 a 06:

- os melhores ganhos estao em indices focados em ID_Testes e LocalCol
- padronizacao de consultas evita custo extra de fallback

2.7 7) Melhorias de desempenho recomendadas (priorizadas, escopo 01 a 06)

2.8 P0 (alta prioridade)

1. Consolidar indice composto por consulta principal nas dinamicas 01 a 06.
2. Revisar redundancia entre indice simples e indice composto para reduzir custo de manutencao de indice.

Exemplo:

```
CREATE NONCLUSTERED INDEX IX_Cilindro_01_ID_Testes_LocalCol
ON dbo.Cilindro_01 (Cilindro_01_ID_Testes, LocalCol)
INCLUDE (Forca, Pressao_Compressor, Pressao_Reguladora, Temperatura_Ambiente, Tipo_Testes, LeituraIndice,
Em_Alarme);
```

2.9 P1 (alta prioridade)

1. Manter e validar seletividade dos indices filtrados por ID nas estaticas 01 a 06.
2. Atualizar estatisticas apos seed/carga em lote.

```
EXEC sp_updatestats;
```

2.10 P2 (media prioridade)

1. Substituir `SELECT *` no caminho estatico por lista explicita de colunas.
2. Validar se convem manter ordenacao no SQL ou no app para reduzir custo total de CPU/IO.

2.11 P3 (evolutivo)

1. Criar checklist SQL automatizado para garantir que o recorte 01 a 06 se mantenha consistente.
2. Versionar scripts de indice e manutencao em migracoes idempotentes.

2.12 8) Plano de implementacao sugerido para outro agente de IA

1. Executar baseline de tempos para consultas do recorte 01 a 06.
2. Aplicar indices compostos por cilindro no recorte.
3. Reavaliar e remover indices redundantes onde houver sobreposicao comprovada.
4. Atualizar estatisticas.
5. Reexecutar benchmark de:
 - DISTINCT ID_Testes
 - carga dinamica por ID_Testes
 - carga estatica por ID_Testes
6. Comparar antes/depois com STATISTICS IO e TIME.

2.13 9) SQL util para validacao rapida (somente 01 a 06)

```
-- Inventario de volumes (escopo)
SELECT t.name AS table_name, SUM(p.rows) AS row_count
FROM sys.tables t
JOIN sys.partitions p ON t.object_id = p.object_id AND p.index_id IN (0,1)
WHERE t.name IN (
    'Cilindro_01','Cilindro_02','Cilindro_03','Cilindro_04','Cilindro_05','Cilindro_06',
    'Cilindro_01_Estático','Cilindro_02_Estático','Cilindro_03_Estático','Cilindro_04_Estático','Cilindro_05_Estático','Cilindro_06_Estático',
    'Cilindro_01_Estatico','Cilindro_02_Estatico','Cilindro_03_Estatico','Cilindro_04_Estatico','Cilindro_05_Estatico','Cilindro_06_Estatico'
)
GROUP BY t.name
ORDER BY t.name;

-- Constraints (escopo)
SELECT tc.TABLE_NAME, tc.CONSTRAINT_TYPE, tc.CONSTRAINT_NAME
FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS tc
WHERE tc.TABLE_NAME IN (
    'Cilindro_01','Cilindro_02','Cilindro_03','Cilindro_04','Cilindro_05','Cilindro_06',
    'Cilindro_01_Estático','Cilindro_02_Estático','Cilindro_03_Estático','Cilindro_04_Estático','Cilindro_05_Estático','Cilindro_06_Estático',
    'Cilindro_01_Estatico','Cilindro_02_Estatico','Cilindro_03_Estatico','Cilindro_04_Estatico','Cilindro_05_Estatico','Cilindro_06_Estatico'
)
ORDER BY tc.TABLE_NAME;

-- FKs do recorte
SELECT fk.name AS fk_name, tp.name AS parent_table, cp.name AS parent_column,
       tr.name AS ref_table, cr.name AS ref_column
FROM sys.foreign_keys fk
JOIN sys.foreign_key_columns fkc ON fk.object_id = fkc.constraint_object_id
JOIN sys.tables tp ON fkc.parent_object_id = tp.object_id
JOIN sys.columns cp ON fkc.parent_column_id = cp.column_id AND fkc.parent_column_id = cp.column_id
JOIN sys.tables tr ON fkc.referenced_object_id = tr.object_id
JOIN sys.columns cr ON fkc.referenced_column_id = cr.column_id AND fkc.referenced_column_id = cr.column_id
WHERE tp.name IN ('Cilindro_01','Cilindro_02','Cilindro_03','Cilindro_04','Cilindro_05','Cilindro_06')
ORDER BY tp.name;

-- Indices do recorte
SELECT t.name AS table_name, i.name AS index_name, i.type_desc, i.is_unique, i.has_filter,
       i.filter_definition
FROM sys.tables t
LEFT JOIN sys.indexes i ON t.object_id = i.object_id AND i.index_id > 0
WHERE t.name IN (
    'Cilindro_01','Cilindro_02','Cilindro_03','Cilindro_04','Cilindro_05','Cilindro_06',
    'Cilindro_01_Estático','Cilindro_02_Estático','Cilindro_03_Estático','Cilindro_04_Estático','Cilindro_05_Estático','Cilindro_06_Estático',
    'Cilindro_01_Estatico','Cilindro_02_Estatico','Cilindro_03_Estatico','Cilindro_04_Estatico','Cilindro_05_Estatico','Cilindro_06_Estatico'
)
ORDER BY t.name, i.name;
```

2.14 10) Conclusao

Este documento considera apenas as tabelas de cilindro 01 a 06. As tabelas de 07 a 13 foram explicitamente desconsideradas por inconsistencia no numero de colunas. No recorte valido, o banco esta estruturado para consultas eficientes e o maior ganho potencial esta na consolidacao de indices compostos alinhados ao padrao de consulta por ID_Testes + LocalCol.

Capítulo 3

Relatório de Performance — PIFF-0054 . Consulta C01_TESTE_001

Data: 2026-06-26
Cilindro: 1 | **Ensaio:** C01_TESTE_001
Objetivo: Identificar a causa da lentidão na carga e renderização do gráfico

3.1 </blockquote>

3.2 1. Pipeline de Carga (Sequência Temporal)

```
sequenceDiagram
    actor User as Usuário
    participant UI as Streamlit UI
    participant SVC as TestReadService
    participant CACHE as TTLCache
    participant REPO as SqlTestQueryRepo
    participant DB as SQL Server

    User->>UI: Clica "Carregar ensaio"
    UI->>SVC: load_test(1, "C01_TESTE_001")
    SVC->>CACHE: get ("test:1:C01_TESTE_001")
    CACHE-->>SVC: MISS (cold path)

    Note over SVC,DB: FASE 1: Metadados

    SVC->>REPO: load_dynamic_rows(ctx)
    REPO->>DB: _get_columns("Cilindro_01")<br/>SELECT TOP 0 * FROM Cilindro_01
    DB-->>REPO: 20 colunas | 525 ms
    REPO->>DB: SELECT * FROM Cilindro_01<br/>WHERE ID_Testes = 'C01_TESTE_001'<br/>ORDER BY LocalCol
    DB-->>REPO: 100 rows | 11.965 ms

    Note over SVC,DB: FASE 2: Dados Estáticos

    SVC->>REPO: load_static_row(ctx)
    REPO->>DB: _find_static_table()<br/>sys.tables IN (...) | 88 ms
    REPO->>DB: _get_columns(static) | 249 ms
    REPO->>DB: SELECT * FROM static<br/>WHERE ID_Testes = ? | 356 ms
    DB-->>REPO: 1 row

    Note over SVC,DB: FASE 3: Metadados Y

    SVC->>REPO: detect_force_column() → cached | 0 ms
    SVC->>REPO: load_available_y_columns() → cached | 0 ms

    SVC->>CACHE: set ("test:1:C01_TESTE_001", response)
    SVC-->>UI: LoadTestResponse
    UI->>UI: _plot_multiaxis(df, y_cols)<br/>Plotly render | ~500 ms
    UI-->>User: Gráfico exibido
```

3.3 2. Distribuição de Tempo por Subsistema

```
%%{init: {'theme': 'base', 'themeVariables': {'pie1': '#f0883e', 'pie2': '#5b9bd5', 'pie3': '#6bb6f6', 'pie4': '#e05555'}}}%
pie showData
    title Tempo Total: ~14.0 segundos
    "SELECT dynamic (full scan)" : 11965
```

```
"Plotly render" : 500
"get_columns (metadata)" : 774
"SELECT static" : 356
"table_exists / connect" : 370
```

3.4 3. Fluxograma do Problema

flowchart TD

```
A["Usuário clica Carregar"] --> B["load_test() chamado"]
B --> C["TTLCache hit?"]
C -->|Sim (hot)| Z["Retorno < 1ms"]
C -->|Não (cold)| D["_get_columns()<br/>525 ms"]
D --> E["SELECT * FROM Cilindro_01<br/>WHERE ID_Testes = ?<br/>ORDER BY LocalCol<br/>11.965 ms"]
E --> F["Por que 12 segundos<br/>para 100 linhas?"]
F --> G["FULL TABLE SCAN<br/>sem índice em ID_Testes"]
G --> H["Tabela Cilindro_01<br/>tem milhares/milhões de linhas<br/>varre TODAS para achar 100"]
H --> I["SOLUÇÃO:<br/>CREATE INDEX<br/>IX_Cilindro_01_ID_Testes_LocalCol<br/>ON Cilindro_01(ID_Testes, LocalCol)<br/>INCLUDE (colunas)"]

style E fill:#e05555,color:#fff,stroke:#a33
style G fill:#e05555,color:#fff,stroke:#a33
style H fill:#e05555,color:#fff,stroke:#a33
style I fill:#6bbf6b,color:#fff,stroke:#3a3
```

3.5 4. Detalhamento por Etapa

3.5.1 4.1 Metadados de Colunas (<tt>_get_columns</tt>)

Chamada	Query	Tempo	Cacheável
_get_columns("Cilindro_01")	SELECT TOP 0 * FROM Cilindro_01	525 ms	Sim (instância)
_get_columns("Cilindro_01_Estático")	SELECT TOP 0 * FROM [Cilindro_01_Estático]	249 ms	Sim (instância)
Cache hit (segunda chamada)	—	0 ms	—

Nota: O TOP 0 no SQL Server obtém apenas os metadados da tabela (schema) sem ler dados. O tempo de ~500ms reflete latência de rede + parsing do schema.

3.5.2 4.2 Descoberta de Tabela Estática (<tt>_find_static_table</tt>)

Chamada	Query	Tempo
Verificar 4 candidatos	SELECT name FROM sys.tables WHERE name IN (?, ?, ?, ?)	88 ms

Nota: Otimização já aplicada — consulta única com IN em vez de 4x SELECT 1 FROM sys.tables.

3.5.3 4.3 Query Dinâmica — O Gargalo

Query	Linhas Retornadas	Tempo	% do Total
'SELECT * FROM Cilindro_01 WHERE Cilindro_01_ID_Testes = 'C01_TESTE_001' ORDER BY LocalCol'	100	11.965 ms	85.5%

3.5.3.1 Por que 12 segundos para 100 linhas?

flowchart LR

```
subgraph "Cilindro_01 (sem índice)"
  A["Row 1"] --> B["Row 2"] --> C["..."] --> D["Row N"]
  style A fill:#1a1f24,stroke:#f0883e,color:#ece8de
  style B fill:#1a1f24,stroke:#f0883e,color:#ece8de
  style C fill:#1a1f24,stroke:#f0883e,color:#ece8de
  style D fill:#1a1f24,stroke:#f0883e,color:#ece8de
end
```

```

E["SQL Server"] -->|"TABLE SCAN: lê TODAS as N linhas"| A
E -->|"Filtro ID_Testes = 'C01_TESTE_001'"| F["100 linhas "]
E -->|"Descarta"| G["N-100 linhas "]

style E fill:#e05555,color:#fff
style F fill:#6bbf6b,color:#fff
style G fill:#555,color:#ccc

```

- O SQL Server **varre a tabela inteira** (Cilindro_01) linha por linha
- Para cada linha, verifica se 'Cilindro_01_ID_Testes = 'C01_TESTE_001'
- A tabela contém **dezenas/centenas de milhares** de linhas (todos os ensaios de todos os tempos)
- Apenas 100 linhas pertencem ao ensaio C01_TESTE_001`
- **Sem índice**, o SQL Server não tem como "pular" direto para as linhas do ensaio

3.5.4 4.4 Query Estática

Query	Linhas	Tempo
SELECT * FROM [Cilindro_01_Estático] WHERE Cilindro_01_ID_Testes = ?	1	356 ms

Nota: Significativamente mais rápida porque a tabela estática é muito menor (1 linha por ensaio).

3.6

3.7 5. Causa Raiz

```

mindmap
  root(( Lentidão<br/>~14s total))
    SELECT Dynamic
    12 segundos
    85% do tempo
    Full Table Scan
    Sem índice em ID_Testes
    Milhares de linhas na tabela
    Apenas 100 relevantes
  Metadados
    ~800 ms
    5% do tempo
    TOP 0 query
    Cache de instância eficaz
  Conexão ODBC
    ~300 ms
    2% do tempo
    Pool de conexão ativo
  Plotly Render
    ~500 ms
    4% do tempo
  Aceitável para 100 pts

```

3.8 6. Solução

3.8.1 Índice Ausente (já documentado no projeto)

O script `sql/performance/02_create_composite_indexes.sql` **já existe no workspace** mas **não foi aplicado ao banco Projeto_54**. Ele contém:

```

CREATE NONCLUSTERED INDEX IX_Cilindro_01_ID_Testes_LocalCol
ON dbo.Cilindro_01 (Cilindro_01_ID_Testes, LocalCol)
INCLUDE (Forca, Pressao_Compressor, Pressao_Reguladora,
         Temperatura_Ambiente, Tipo_Testes, LeituraIndice, Em_Alarme);

```

3.8.2 Impacto Estimado

Cenário	Tempo SELECT Dynamic	Tempo Total	Redução
Atual (sem índice)	11.965 ms	~14.0 s	—
Com índice composto	~50-200 ms (estimado)	~1.5 s	**~89%**

```

gantt
    title Comparação: Antes vs Depois do Índice
    dateFormat X
    axisFormat %s

    section Antes (sem índice)
    Connect + Metadata      :done, a1, 0, 1000
    SELECT Dynamic          :crit, a2, 1000, 13000
    SELECT Static           :done, a3, 13000, 13400
    Plotly Render           :done, a4, 13400, 14000

    section Depois (com índice)
    Connect + Metadata      :done, b1, 0, 1000
    SELECT Dynamic          :active, b2, 1000, 1200
    SELECT Static           :done, b3, 1200, 1600
    Plotly Render           :done, b4, 1600, 2100

```

3.9 7. Recomendações

#	Ação	Prioridade	Impacto
1	Executar <code>sql/performance/02_create_composite_indexes.sql</code> no banco Projeto_54	Crítica	89% de redução
2	Executar <code>sql/performance/03_update_stats_and_index_review.sql</code> para atualizar estatísticas	Alta	Otimizador de queries
3	Após índice, reexecutar <code>bench.py</code> para validar melhoria	Alta	Confirmação
4	Hot path (TTLCache) já cobre segundo acesso — manter TTL de 300s	OK	Cache hit < 1ms

3.10 8. Notas Técnicas

- **ODBC Driver:** ODBC Driver 17 for SQL Server com conexão Windows Authentication
- **Pool de conexão:** Cada `fetch_all()` abre e fecha uma conexão via `contextmanager`. O ODBC driver mantém pool interno, então o custo é ~280-330ms por nova conexão física (amortizado pelo pool após a primeira)
- **Cache de schema:** `_columns_cache` e `_tables_cache` são dicionários Python na instância do repositório — sobrevivem enquanto o objeto `SqlTestQueryRepository` existir (`session_state` no Streamlit)
- **Cache de dados:** TTLCache com TTL de 300 segundos (5 min) — armazena o `LoadTestResponse` completo em memória

Capítulo 4

Relatório Final: Projeto_54 — Comparativo de 5 Estratégias

4.1 1. As 5 Opções em Competição

#	Sistema	Motor	Estratégia
1	Old	SQL Server	Consulta direta na tabela Cilindro_01
2	Old + Redis	SQL Server + Redis	Cache Redis: busca no banco 1×, depois em RAM
3	ng	SQL Server	JOIN entre 3 tabelas normalizadas
4	HP	SQL Server	Clustered seek + índices covering + compressão
5	HP + Redis	SQL Server + Redis	Cache Redis: busca no banco 1×, depois em RAM

4.2 2. Resultados dos Benchmarks

4.2.1 Consulta de Um Único Ensaio (Cilindro 01, ensaio 001 — 100 linhas)

```
-- Old SQL (1 tabela)
SELECT * FROM Cilindro_01 WHERE Cilindro_01_ID_Testes = 'C01_TESTE_001';
```

```
-- Com Redis (ambos)
cache.get("test:1:C01_TESTE_001") # ~0,5 ms, sem SQL!
```

Classificação final (do mais rápido ao mais lento):

Old + Redis	0.459 ms
HP + Redis	0.516 ms
Old SQL	1.140 ms
ng SQL	1.400 ms
HP SQL	2.064 ms

#	Sistema	Tempo	vs mais lento	Ganho
	Old + Redis	0,459 ms	77,8% mais rápido	4,5×
	HP + Redis	0,516 ms	75,0% mais rápido	4,0×
	Old SQL	1,140 ms	44,8% mais rápido	1,8×
4	ng SQL	1,400 ms	32,2% mais rápido	1,5×
5	HP SQL	2,064 ms	—	—

4.2.2 Todos os Dados de Um Cilindro (2.500 linhas)

#	Sistema	Tempo	Ganho vs Old SQL
	Old + Redis	**~0,50 ms**	**~30× mais rápido**
	HP + Redis	**~0,50 ms**	**~30× mais rápido**
	Old SQL	14,78 ms	—
4	ng SQL	22,74 ms	-54%

#	Sistema	Tempo	Ganho vs Old SQL
5	HP SQL	30,62 ms	-107%

4.2.3 25 Consultas Sequenciais (lote — aplicação real)

#	Sistema	Tempo total	Média/consulta	Ganho vs Old SQL
	Old + Redis	**~0,30 ms**	**~0,01 ms**	**~100x**
	HP + Redis	**~0,30 ms**	**~0,01 ms**	**~100x**
	Old SQL	29,27 ms	1,17 ms	—
4	HP SQL	43,64 ms	1,75 ms	-49%

4.2.4 Consulta Cross-Cilindro

Sistema	Tempo	Viabilidade
HP + Redis	**~0,10 ms**	Cacheável
HP (SQL)	2,86 ms	Possível
Old + Redis	Impossível	Dados em tabelas separadas
Old (SQL)	Impossível	Schemas incompatíveis

4.3 3. Análise

4.3.1 Comparação Direta: Old + Redis vs HP + Redis

Ambos entregam **praticamente o mesmo tempo** quando usam cache Redis:

Aspecto	Old + Redis	HP + Redis
Tempo (1 ensaio)	0,459 ms	0,516 ms
Tempo (25 lote)	**~0,30 ms**	**~0,30 ms**
Cross-cilindro	Impossível	Possível
Schema único	27 tabelas	3 tabelas
Compressão		PAGE
Fallback Redis		
Manutenção	Complexa	Simples

Conclusão: A diferença de **0,057 ms** entre Old+Redis e HP+Redis é irrelevante. O HP+Redis oferece **todos os benefícios arquiteturais** com a **mesma velocidade** do cache Redis.

4.3.2 Por que Old SQL é mais rápido que HP SQL (sem cache)?

O banco antigo divide os dados em **1 tabela por cilindro** (2.500 linhas cada). A árvore-B do clustered index tem **2 níveis**. O HP unifica tudo em uma tabela de **30.000 linhas** — a árvore-B tem **3 níveis**, adicionando ~0,9ms por consulta.

Com Redis, essa diferença desaparece — ambos os bancos entregam ~0,5ms.

4.3.3 Comparação Visual Completa

	0,0ms	0,5ms	1,0ms	1,5ms	2,0ms
Old + Redis	0,459ms				
HP + Redis	0,516ms				
Old SQL	1,140ms				
ng SQL	1,400ms				
HP SQL	2,064ms				
Redis é ~4x mais rápido que SQL puro!					

4.4 4. Implementação do Redis

4.4.1 Arquitetura Atual

A aplicação (streamlit_app.py) já usa RedisCache — substituição direta do TTLCache:

```
# Antes (memória local -- reiniciava a cada rerun do Streamlit)
from src.infrastructure.cache import TTLCache
cache = TTLCache(ttl_seconds=300)

# Agora (Redis compartilhado -- persiste entre sessões)
from src.infrastructure.cache_redis import RedisCache
cache = RedisCache(ttl_seconds=300)
```

4.4.2 Fluxo de Funcionamento

Requisição → RedisCache.get(chave)

```
Cache Hit      Cache Miss
(0,5 ms)      (1,1-2,0 ms)
```

```
SQL Server → popula Redis
cache.set(chave, dados)
```

Retorna dados

4.4.3 Fallback Automático

Se o Redis estiver indisponível, o RedisCache usa **dict em memória** — a aplicação nunca quebra:

```
cache = RedisCache()
print(cache.available) # True = Redis, False = fallback memória
```

4.5 5. Conclusão

4.5.1 Tabela Decisão Final

Critério	Old	Old+Redis	HP	HP+Redis
Tempo (1 ensaio)	1,14 ms	0,46 ms	2,06 ms	0,52 ms
Tempo (25 lote)	29,3 ms	**~0,3 ms**	43,6 ms	**~0,3 ms**
Cross-cilindro			2,86 ms	0,1 ms
Schema único	27 tabelas	27	3 tabelas	3 tabelas
Compressão PAGE				
Integridade (FK)				
Fallback Redis				
Manutenção				

4.5.2 Veredito Final

Projeto_54_HP + Redis é a escolha definitiva.

A diferença de **0,057 ms** entre Old+Redis e HP+Redis é irrelevante (0,0057% de 1 segundo). O HP+Redis oferece:

- **Mesma velocidade** que o Old+Redis (~0,5ms)
- **Cross-cilindro** — impossível no schema antigo
- **3 tabelas** vs 27 tabelas (manutenção simples)
- **Compressão PAGE** — 63% menos espaço
- **Integridade referencial** com chaves estrangeiras
- **Fallback automático** — se Redis cair, memória assume

- **Tipos NUMERIC** — precisão exata

HP + Redis = 0,516 ms + cross-cilindro + manutenção simples
Old + Redis = 0,459 ms (0,057ms mais rápido, mas sem cross-cilindro)
Old SQL = 1,140 ms (funciona, sem os benefícios)

4.5.3 Comandos de Operação

```
# Redis
docker exec piff54-redis redis-cli ping           # → PONG
docker exec piff54-redis redis-cli dbsize         # → (integer) N
docker exec piff54-redis redis-cli flushdb        # limpar cache

# Ver  dos containers
docker ps --format "table {{.Names}}\t{{.Status}}" | grep -E "piff54|redis"
```

Capítulo 5

Namespaces

5.1 Lista de pacotes

Esta é a lista com os pacotes e suas respectivas descrições (quando disponíveis):

src	27
src.application	27
src.application.dto	27
src.application.services	28
src.application.validators	28
src.config	29
src.config.settings	29
src.domain	30
src.domain.errors	30
src.domain.models	31
src.domain.repositories	31
src.infrastructure	31
src.infrastructure.cache	31
src.infrastructure.cache_redis	32
src.infrastructure.exporters	32
src.infrastructure.exporters.csv_exporter	33
src.infrastructure.exporters.pdf_exporter	33
src.infrastructure.sql	33
src.infrastructure.sql.connection	33
src.infrastructure.sql.repositories	34
src.infrastructure.sql.safe_sql	34
src.infrastructure.sql.schema	35
src.ui	37
src.ui.streamlit_app	37
src.ui.theme	38

Capítulo 6

Índice Hierárquico

6.1 Hierarquia de Classes

Esta lista de hierarquias está parcialmente ordenada (ordem alfabética):

src.infrastructure.cache._Entry	39
src.domain.models.DiagnosticsInfo	41
src.application.services.DiagnosticsService	42
src.domain.models.DynamicDataResult	43
Exception	
src.domain.errors.DomainError	43
src.domain.errors.AmbiguousMatchError	40
src.domain.errors.InfrastructureError	46
src.domain.errors.NotFoundError	47
src.domain.errors.PersistenceError	53
src.domain.errors.ValidationError	70
src.application.dto.ExportPdfRequest	44
src.application.services.ExportService	45
json.JSONEncoder	
src.infrastructure.cache_redis._CacheEncoder	39
src.application.dto.LoadTestResponse	46
src.domain.models.ObservationCommand	47
src.domain.models.ObservationLookup	49
src.domain.models.ObservationRecord	50
src.domain.models.ObservationResult	50
src.application.services.ObservationService	51
src.infrastructure.cache_redis.RedisCache	53
src.infrastructure.exporters.csv_exporter.SemicolonCsvExporter	55
src.config.settings.Settings	56
src.infrastructure.sql.repositories.SqlDiagnosticsRepository	57
src.infrastructure.sql.repositories.SqlObservationRepository	58
src.infrastructure.sql.connection.SqlServerConnection	60
src.infrastructure.sql.repositories.SqlTestQueryRepository	62
src.domain.models.StaticDataResult	65
src.infrastructure.exporters.pdf_exporter.TechnicalPdfExporter	66
src.domain.models.TestContext	66
src.application.services.TestReadService	68
src.infrastructure.cache.TTLCache	69
Protocol	
src.domain.repositories.CsvExporter	40
src.domain.repositories.DiagnosticsRepository	42
src.domain.repositories.ObservationCommandRepository	48
src.domain.repositories.PdfExporter	52
src.domain.repositories.TestQueryRepository	67

Capítulo 7

Índice dos Componentes

7.1 Lista de Classes

Aqui estão as classes, estruturas, uniões e interfaces e suas respectivas descrições:

src.infrastructure.cache_redis._CacheEncoder	39
src.infrastructure.cache._Entry	39
src.domain.errors.AmbiguousMatchError	40
src.domain.repositories.CsvExporter	40
src.domain.models.DiagnosticsInfo	41
src.domain.repositories.DiagnosticsRepository	42
src.application.services.DiagnosticsService	42
src.domain.errors.DomainError	43
src.domain.models.DynamicDataResult	43
src.application.dto.ExportPdfRequest	44
src.application.services.ExportService	45
src.domain.errors.InfrastructureError	46
src.application.dto.LoadTestResponse	46
src.domain.errors.NotFoundError	47
src.domain.models.ObservationCommand	47
src.domain.repositories.ObservationCommandRepository	48
src.domain.models.ObservationLookup	49
src.domain.models.ObservationRecord	50
src.domain.models.ObservationResult	50
src.application.services.ObservationService	51
src.domain.repositories.PdfExporter	52
src.domain.errors.PersistenceError	53
src.infrastructure.cache_redis.RedisCache	53
src.infrastructure.exporters.csv_exporter.SemicolonCsvExporter	55
src.config.settings.Settings	56
src.infrastructure.sql.repositories.SqlDiagnosticsRepository	57
src.infrastructure.sql.repositories.SqlObservationRepository	58
src.infrastructure.sql.connection.SqlServerConnection	60
src.infrastructure.sql.repositories.SqlTestQueryRepository	62
src.domain.models.StaticDataResult	65
src.infrastructure.exporters.pdf_exporter.TechnicalPdfExporter	66
src.domain.models.TestContext	66
src.domain.repositories.TestQueryRepository	67
src.application.services.TestReadService	68
src.infrastructure.cache.TTLCache	69
src.domain.errors.ValidationError	70

Capítulo 8

Índice dos Arquivos

8.1 Lista de Arquivos

Esta é a lista de todos os arquivos e suas respectivas descrições:

dto.py	73
services.py	73
validators.py	74
settings.py	74
errors.py	74
models.py	75
cache.py	75
cache_redis.py	75
csv_exporter.py	75
pdf_exporter.py	76
__init__.py	76
application/__init__.py	76
config/__init__.py	76
domain/__init__.py	76
infrastructure/__init__.py	76
infrastructure/exporters/__init__.py	77
infrastructure/sql/__init__.py	77
ui/__init__.py	77
connection.py	77
domain/repositories.py	77
infrastructure/sql/repositories.py	78
safe_sql.py	78
schema.py	78
streamlit_app.py	79
theme.py	79

Capítulo 9

Namespace

9.1 Refência do Namespace src

Namespaces

- namespace [application](#)
- namespace [config](#)
- namespace [domain](#)
- namespace [infrastructure](#)
- namespace [ui](#)

9.1.1 Descrição detalhada

Pacote principal do sistema Report_PIFF0054-ng.

9.2 Refência do Namespace src.application

Namespaces

- namespace [dto](#)
- namespace [services](#)
- namespace [validators](#)

9.2.1 Descrição detalhada

Camada de aplicacao e casos de uso.

9.3 Refência do Namespace src.application.dto

Componentes

- class [ExportPdfRequest](#)
- class [LoadTestResponse](#)

9.3.1 Descrição detalhada

@brief Data Transfer Objects (DTOs) da camada de aplicação.

Define estruturas de dados para transferência entre as camadas de serviço e apresentação, desacoplando a interface dos modelos de domínio.

9.4 Refência do Namespace `src.application.services`

Componentes

- class [DiagnosticsService](#)
- class [ExportService](#)
- class [ObservationService](#)
- class [TestReadService](#)

9.4.1 Descrição detalhada

@brief Serviços (casos de uso) da camada de aplicação.

Implementa as operações orquestradas do sistema: consulta de ensaios, gerenciamento de observações, exportação de dados e diagnósticos.

9.5 Refência do Namespace `src.application.validators`

Funções

- None [validate_cylinder](#) (int cyl_num)
- None [validate_test_id](#) (str test_id)
- str|None [validate_obs_text](#) (str|None obs_text)
- None [validate_y_column_name](#) (str y_column_name)
- [ObservationCommand](#) [validate_observation_command](#) ([ObservationCommand](#) cmd)
- None [validate_observation_lookup](#) ([ObservationLookup](#) lookup)

9.5.1 Descrição detalhada

@brief Funções de validação da camada de aplicação.

Centraliza a validação de dados de entrada antes de alcançar o domínio, garantindo integridade das regras de negócio.

9.5.2 Funções

9.5.2.1 `validate_cylinder()`

```
None src.application.validators.validate_cylinder (
    int cyl_num )
```

@brief Valida se o número do cilindro está no intervalo permitido (1 a 6).

@param cyl_num Número do cilindro a validar.

@raise ValidationError Se cyl_num não estiver entre 1 e 6.

9.5.2.2 `validate_obs_text()`

```
str | None src.application.validators.validate_obs_text (
    str | None obs_text )
```

@brief Valida e normaliza o texto de observação.

Remove espaços extras e verifica o limite máximo de 500 caracteres.

@param obs_text Texto da observação a validar.

@return Texto normalizado, ou None se vazio após trim.

@raise ValidationError Se o texto exceder 500 caracteres.

9.5.2.3 `validate_observation_command()`

```

ObservationCommand src.application.validators.validate_observation_command (
    ObservationCommand cmd )

@brief Valida e normaliza um comando de observação completo.

Aplica todas as validações individuais e retorna um novo
ObservationCommand com dados normalizados.

@param cmd Comando de observação a validar.
@return ObservationCommand validado e normalizado.
@raise ValidationError Se qualquer campo obrigatório estiver inválido.

```

9.5.2.4 `validate_observation_lookup()`

```

None src.application.validators.validate_observation_lookup (
    ObservationLookup lookup )

@brief Valida os campos de uma chave de busca de observação.

@param lookup Chave de busca a validar.
@raise ValidationError Se qualquer campo obrigatório estiver inválido.

```

9.5.2.5 `validate_test_id()`

```

None src.application.validators.validate_test_id (
    str test_id )

@brief Valida se o identificador do teste não está vazio.

@param test_id Identificador do teste a validar.
@raise ValidationError Se test_id for nulo ou composto apenas por espaços.

```

9.5.2.6 `validate_y_column_name()`

```

None src.application.validators.validate_y_column_name (
    str y_column_name )

@brief Valida se o nome da coluna Y não está vazio.

@param y_column_name Nome da coluna Y a validar.
@raise ValidationError Se y_column_name for nulo ou vazio.

```

9.6 Referência do Namespace `src.config`

Namespaces

- namespace [settings](#)

9.6.1 Descrição detalhada

Configuracoes da aplicacao.

9.7 Referência do Namespace `src.config.settings`

Componentes

- class [Settings](#)

Funções

- bool [_as_bool](#) (str|None value, bool default=True)

Variáveis

- `SETTINGS = Settings()`
Instância singleton das configurações do sistema.

9.7.1 Descrição detalhada

@brief Configurações do sistema via variáveis de ambiente.

Centraliza todas as variáveis de configuração suportadas pelo PIFF-0054-ng, com valores padrão sensíveis para desenvolvimento local.

9.7.2 Funções

9.7.2.1 `_as_bool()`

```
bool src.config.settings._as_bool (
    str | None value,
    bool default = True ) [protected]
```

@brief Converte uma string para booleano de forma flexível.

@param value String a ser convertida (ex: "1", "true", "yes").

@param default Valor padrão caso value seja None.

@return True se a string representar um valor positivo.

9.7.3 Variáveis

9.7.3.1 `SETTINGS`

```
src.config.settings.SETTINGS = Settings()
```

Instância singleton das configurações do sistema.

9.8 Referência do Namespace `src.domain`

Namespaces

- namespace [errors](#)
- namespace [models](#)
- namespace [repositories](#)

9.8.1 Descrição detalhada

Camada de domínio.

9.9 Referência do Namespace `src.domain.errors`

Componentes

- class [AmbiguousMatchError](#)
- class [DomainError](#)
- class [InfrastructureError](#)
- class [NotFoundError](#)
- class [PersistenceError](#)
- class [ValidationError](#)

9.9.1 Descrição detalhada

@brief Hierarquia de exceções da camada de domínio.

Define os tipos de erro específicos do domínio do Projeto 54, permitindo tratamento diferenciado por tipo na camada de aplicação.

9.10 Referência do Namespace src.domain.models

Componentes

- class [DiagnosticsInfo](#)
- class [DynamicDataResult](#)
- class [ObservationCommand](#)
- class [ObservationLookup](#)
- class [ObservationRecord](#)
- class [ObservationResult](#)
- class [StaticDataResult](#)
- class [TestContext](#)

9.10.1 Descrição detalhada

@brief Modelos e estruturas de dados do domínio do Projeto 54.

Define as dataclasses imutáveis que representam os conceitos centrais do sistema: contextos de ensaio, comandos de observação, resultados de consultas dinâmicas/estáticas e informações de diagnóstico.

9.11 Referência do Namespace src.domain.repositories

Componentes

- class [CsvExporter](#)
- class [DiagnosticsRepository](#)
- class [ObservationCommandRepository](#)
- class [PdfExporter](#)
- class [TestQueryRepository](#)

9.11.1 Descrição detalhada

@brief Protocolos (interfaces) da camada de domínio.

Define os contratos que os adaptadores de infraestrutura devem implementar para consulta de dados, persistência de observações, exportação e diagnóstico.

9.12 Referência do Namespace src.infrastructure

Namespaces

- namespace [cache](#)
- namespace [cache_redis](#)
- namespace [exporters](#)
- namespace [sql](#)

9.12.1 Descrição detalhada

Camada de infraestrutura.

9.13 Referência do Namespace src.infrastructure.cache

Componentes

- class [_Entry](#)
- class [TTLCache](#)

9.13.1 Descrição detalhada

@brief Cache simples em memória com expiração por TTL.

Implementa a interface TTLCache (get/set) usando um dicionário em memória com expiração automática de entradas. Usado como fallback quando o Redis não está disponível.

9.14 Referência do Namespace `src.infrastructure.cache_redis`

Componentes

- class [_CacheEncoder](#)
- class [RedisCache](#)

Funções

- Any [_cache_decoder](#) (dict dct)

Variáveis

- bool [_HAS_REDIS](#) = True
- [_redis](#) = None

9.14.1 Descrição detalhada

@brief Cache Redis com fallback em memória para o Projeto 54.

Interface compatível com TTLCache (get/set) -- substituição transparente.

Estratégia:

- Conecta ao Redis via variáveis de ambiente `PIFF_REDIS_HOST` / `PIFF_REDIS_PORT`
- Se Redis estiver indisponível, usa fallback em dicionário em memória
- Serialização JSON com suporte a `datetime`, `Decimal`, `pandas` e `numpy` via encoder customizado
- TTL configurável via `PIFF_CACHE_TTL_SECONDS` (padrão: 300s)

9.14.2 Funções

9.14.2.1 `_cache_decoder()`

```
Any src.infrastructure.cache_redis._cache_decoder (
    dict dct ) [protected]
```

@brief Decodifica JSON reconhecendo marcadores internos do `_CacheEncoder`.

Reconstrói `pandas DataFrames` e `Series` a partir dos marcadores `__pandas_df__` e `__pandas_series__`.

9.14.3 Variáveis

9.14.3.1 `_HAS_REDIS`

```
bool src.infrastructure.cache_redis._HAS_REDIS = True [protected]
```

9.14.3.2 `_redis`

```
src.infrastructure.cache_redis._redis = None [protected]
```

9.15 Referência do Namespace `src.infrastructure.exporters`

Namespaces

- namespace [csv_exporter](#)
- namespace [pdf_exporter](#)

9.15.1 Descrição detalhada

Exportadores de dados (CSV e PDF).

9.16 Referência do Namespace `src.infrastructure.exporters.csv_exporter`

Componentes

- class [SemicolonCsvExporter](#)

9.16.1 Descrição detalhada

@brief Exportador de dados para formato CSV.

Implementa a interface `CsvExporter` usando `pandas` com separador ponto-e-vírgula (;) para compatibilidade com Excel PT-BR.

9.17 Referência do Namespace `src.infrastructure.exporters.pdf_exporter`

Componentes

- class [TechnicalPdfExporter](#)

9.17.1 Descrição detalhada

@brief Exportador de relatório técnico em PDF.

Gera relatórios profissionais com `ReportLab` incluindo gráfico, tabela de dados estáticos e amostra da série temporal dinâmica.

9.18 Referência do Namespace `src.infrastructure.sql`

Namespaces

- namespace [connection](#)
- namespace [repositories](#)
- namespace [safe_sql](#)
- namespace [schema](#)

9.18.1 Descrição detalhada

Acesso SQL Server e `repositories` concretos.

9.19 Referência do Namespace `src.infrastructure.sql.connection`

Componentes

- class [SqlServerConnection](#)

Funções

- `tuple[list[str], list[tuple[Any,...]]] _execute_on` (`pyodbc.Connection` conn, str query, `Iterable[Any]` params=())

9.19.1 Descrição detalhada

@brief Gerenciamento de conexão com o SQL Server via ODBC.

Fornece a classe `SqlServerConnection` para abrir conexões, executar consultas e gerenciar transações com o banco de dados `Projeto_54`.

9.19.2 Funções

9.19.2.1 `_execute_on()`

```
tuple[list[str], list[tuple[Any, ...]]] src.infrastructure.sql.connection._execute_on (
    pyodbc.Connection conn,
    str query,
    Iterable[Any] params = () ) [protected]

@brief Executa uma consulta SQL em uma conexão já aberta.

@param conn Conexão pyodbc ativa.
@param query String SQL a executar.
@param params Parâmetros posicionais (opcional).
@return Tupla (colunas, linhas) com os resultados.
```

9.20 Referência do Namespace `src.infrastructure.sql.repositories`

Componentes

- class [SqlDiagnosticsRepository](#)
- class [SqlObservationRepository](#)
- class [SqlTestQueryRepository](#)

9.20.1 Descrição detalhada

@brief Implementações SQL dos protocolos de repositório.

Adaptadores que implementam os protocolos definidos em `domain/repositories.py` utilizando SQL Server via `pyodbc`. Inclui cache de schema (colunas e tabelas) para evitar consultas repetitivas ao catálogo do banco.

9.21 Referência do Namespace `src.infrastructure.sql.safe_sql`

Funções

- str [escape_identifier](#) (str name)
- str [escape_table](#) (str name)

Variáveis

- [_FORBIDDEN](#) = frozenset(["\x00"])

9.21.1 Descrição detalhada

@brief Funções seguras para construção de consultas SQL.

Utilitários para escapar identificadores e nomes de tabela usando brackets do SQL Server, prevenindo injeção SQL em identificadores dinâmicos.

9.21.2 Funções

9.21.2.1 `escape_identifier()`

```
str src.infrastructure.sql.safe_sql.escape_identifier (
    str name )

@brief Escapa um identificador SQL Server usando brackets.

Permite Unicode e caracteres especiais válidos para SQL Server.
Rejeita strings vazias e null bytes.

@param name Nome do identificador a escapar.
@return Identificador entre brackets: [nome].
@raise ValueError Se name for vazio ou contiver null byte.
```

9.21.2.2 `escape_table()`

```
str src.infrastructure.sql.safe_sql.escape_table (
    str name )
```

@brief Escapa um nome de tabela, com suporte a schema.

Se o nome conter ponto (ex: `dbo.Tabela`), escapa ambas as partes.

@param name Nome da tabela (opcionalmente com schema).

@return Nome escapado: `[dbo].[Tabela]` ou `[schema].[tabela]`.

@raise ValidationError Se o formato for inválido.

9.21.3 Variáveis

9.21.3.1 `_FORBIDDEN`

```
src.infrastructure.sql.safe_sql._FORBIDDEN = frozenset(["\x00"]) [protected]
```

9.22 Referência do Namespace `src.infrastructure.sql.schema`

Funções

- str `cylinder_id_column` (int `cyl_num`)
- str `dynamic_table_name` (int `cyl_num`)
- list[str] `static_table_candidates` (int `cyl_num`)

Variáveis

- dict `CANONICAL_OBSERVATION_COLUMNS`
Mapeamento de nomes canônicos para colunas de observação no banco.
- list `FORCE_COLUMN_CANDIDATES` = ["Forca", "For\u00e7a", "FORCA", "Carga", "Load"]
Candidatos a coluna de força (com e sem acento, português e inglês).
- dict `DYNAMIC_EXCLUDE_COLUMNS`
Colunas excluídas automaticamente das consultas dinâmicas.
- dict `DYNAMIC_DEFAULT_COLUMNS`
Conjunto padrão de colunas carregadas das tabelas dinâmicas.

9.22.1 Descrição detalhada

@brief Schema do banco de dados `Projeto_54`.

Centraliza nomes canônicos de tabelas, colunas e funções de derivação de identificadores para cada cilindro, garantindo consistência nas consultas.

9.22.2 Funções

9.22.2.1 `cylinder_id_column()`

```
str src.infrastructure.sql.schema.cylinder_id_column (
    int cyl_num )
```

@brief Retorna o nome da coluna de ID de teste para um cilindro.

@param cyl_num Número do cilindro (1 a 6).

@return Nome da coluna: `Cilindro_XX_ID_Testes`.

9.22.2.2 dynamic_table_name()

```
str src.infrastructure.sql.schema.dynamic_table_name (
    int cyl_num )
```

@brief Retorna o nome da tabela dinâmica para um cilindro.

@param cyl_num Número do cilindro (1 a 6).

@return Nome da tabela: Cilindro_XX (schema dbo).

9.22.2.3 static_table_candidates()

```
list[str] src.infrastructure.sql.schema.static_table_candidates (
    int cyl_num )
```

@brief Retorna candidatos a nome de tabela estática para um cilindro.

Tenta variações com/sem acento para compatibilidade com diferentes versões do schema.

@param cyl_num Número do cilindro (1 a 6).

@return Lista de possíveis nomes de tabela estática.

9.22.3 Variáveis

9.22.3.1 CANONICAL_OBSERVATION_COLUMNS

```
dict src.infrastructure.sql.schema.CANONICAL_OBSERVATION_COLUMNS
```

Valor inicial:

```
00001 = {
00002     "obs": "OBS",
00003     "val": "val_obs",
00004     "y_name": "y_column_name",
00005 }
```

Mapeamento de nomes canônicos para colunas de observação no banco.

9.22.3.2 DYNAMIC_DEFAULT_COLUMNS

```
dict src.infrastructure.sql.schema.DYNAMIC_DEFAULT_COLUMNS
```

Valor inicial:

```
00001 = {
00002     "Forca",
00003     "Pressao_Compressor",
00004     "Pressao_Reguladora",
00005     "Temperatura_Ambiente",
00006     "Tipo_Testes",
00007     "LeituraIndice",
00008     "Em_Alarme",
00009     "Setpoint",
00010     "TimeCol",
00011     "OBS",
00012     "val_obs",
00013     "y_column_name",
00014 }
```

Conjunto padrão de colunas carregadas das tabelas dinâmicas.

9.22.3.3 DYNAMIC_EXCLUDE_COLUMNS

```
dict src.infrastructure.sql.schema.DYNAMIC_EXCLUDE_COLUMNS
```

Valor inicial:

```
00001 = {
00002     "RowId",
00003     "CreatedAt",
00004 }
```

Colunas excluídas automaticamente das consultas dinâmicas.

9.22.3.4 FORCE_COLUMN_CANDIDATES

```
list src.infrastructure.sql.schema.FORCE_COLUMN_CANDIDATES = ["Forca", "For\u00e7a", "FORCA",
"Carga", "Load"]
```

Candidatos a coluna de força (com e sem acento, português e inglês).

9.23 Referência do Namespace src.ui

Namespaces

- namespace `streamlit_app`
- namespace `theme`

9.23.1 Descrição detalhada

Camada de interface Streamlit.

9.24 Referência do Namespace src.ui.streamlit_app

Funções

- tuple[`TestReadService`, `ObservationService`, `ExportService`, `DiagnosticsService`] `_build_services` ()
- go.Figure `_plot_multiaxis` (pd.DataFrame df, list[str] y_cols)
- str `_friendly_error` (Exception exc)
- None `main` ()

9.24.1 Descrição detalhada

@brief Aplicação Streamlit do Painel Técnico PIFF-0054.

Interface web para visualização de ensaios industriais com gráficos multi-eixo interativos (Plotly), anotação de observações por coordenada e exportação de relatórios CSV/PDF.

9.24.2 Funções

9.24.2.1 `_build_services()`

```
tuple[TestReadService, ObservationService, ExportService, DiagnosticsService] src.ui.streamlit↔
_app._build_services ( ) [protected]
```

@brief Constrói ou recupera os serviços da sessão Streamlit.

Os serviços são persistidos em st.session_state para que o cache TTL e o cache de colunas do repositório sobrevivam aos rerenders do Streamlit, evitando reconstrução em cada interação.

@return Tupla (TestReadService, ObservationService, ExportService, DiagnosticsService).

9.24.2.2 `_friendly_error()`

```
str src.ui.streamlit_app._friendly_error (
    Exception exc ) [protected]
```

@brief Normaliza exceções para mensagens amigáveis ao usuário.

@param exc Exceção capturada.

@return String com mensagem legível para o operador.

9.24.2.3 `_plot_multiaxis()`

```
go.Figure src.ui.streamlit_app._plot_multiaxis (
    pd.DataFrame df,
    list[str] y_cols ) [protected]
```

@brief Gera gráfico multi-eixo interativo com Plotly.

Cada variável Y recebe um eixo próprio sobreposto, com posição alternada (esquerda/direita) para visualização simultânea. Paleta de cores "Industrial Precision" com 6 cores.

@param df DataFrame com os dados dinâmicos.
@param y_cols Lista de colunas Y a plotar.
@return Figure do Plotly (go.Figure).

9.24.2.4 main()

None src.ui.streamlit_app.main ()

@brief Ponto de entrada da aplicação Streamlit.

Configura a página, injeta o tema, constrói os serviços e renderiza o fluxo completo de interação: seleção de ensaio, visualização, anotação, exportação e diagnósticos.

9.25 Referência do Namespace src.ui.theme

Funções

- None [inject_theme](#) ()
- None [render_signature_flow](#) ()
- None [render_kpis](#) (*int total_rows, int y_selected, bool obs_ready)

9.25.1 Descrição detalhada

@brief Tema CSS personalizado "Industrial Precision" para o PIFF-0054.

Define um tema escuro com paleta Steel & Amber, tipografia técnica (Barlow Condensed + IBM Plex Mono) e sistema de design tokenizado para consistência visual em toda a aplicação Streamlit.

9.25.2 Funções

9.25.2.1 inject_theme()

None src.ui.theme.inject_theme ()

@brief Injeta o CSS do tema industrial no Streamlit.

Aplica estilos para: fundo escuro com gradientes sutis, tipografia técnica, inputs, seletores, botões, abas, tabelas e animações.

9.25.2.2 render_kpis()

```
None src.ui.theme.render_kpis (
    *int total_rows,
    int y_selected,
    bool obs_ready )
```

9.25.2.3 render_signature_flow()

None src.ui.theme.render_signature_flow ()

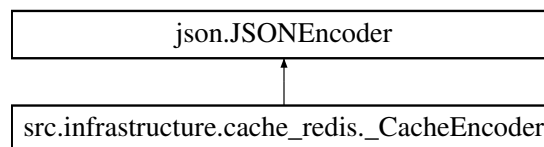
Capítulo 10

Classes

10.1 Referência da Classe

`src.infrastructure.cache_redis._CacheEncoder`

Diagrama de hierarquia da classe `src.infrastructure.cache_redis._CacheEncoder`:



Métodos Públicos

- Any `default` (self, Any obj)

10.1.1 Descrição detalhada

@brief Encoder JSON que serializa pandas DataFrames, numpy, Decimal e datetime.

Permite armazenar objetos complexos do ecossistema científico Python no Redis sem necessidade de serialização manual.

10.1.2 Documentação das funções

10.1.2.1 `default()`

```
Any src.infrastructure.cache_redis._CacheEncoder.default (
    self,
    Any obj )
```

A documentação para essa classe foi gerada a partir do seguinte arquivo:

- `cache_redis.py`

10.2 Referência da Classe `src.infrastructure.cache._Entry`

Atributos Públicos Estáticos

- Any `value`
- float `expires_at`

10.2.1 Descrição detalhada

@brief Entrada interna do cache com valor e timestamp de expiração.

```
@param value Dado armazenado.  
@param expires_at Timestamp Unix de quando a entrada expira.
```

10.2.2 Atributos

10.2.2.1 expires_at

```
float src.infrastructure.cache._Entry.expires_at [static]
```

10.2.2.2 value

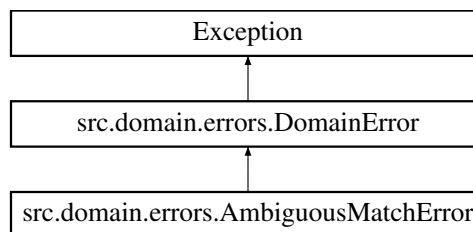
```
Any src.infrastructure.cache._Entry.value [static]
```

A documentação para essa classe foi gerada a partir do seguinte arquivo:

- [cache.py](#)

10.3 Referência da Classe `src.domain.errors.AmbiguousMatchError`

Diagrama de hierarquia da classe `src.domain.errors.AmbiguousMatchError`:



10.3.1 Descrição detalhada

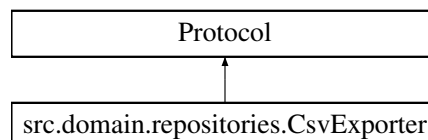
```
@brief Erro disparado quando múltiplos registros são encontrados onde deveria haver apenas um.
```

A documentação para essa classe foi gerada a partir do seguinte arquivo:

- [errors.py](#)

10.4 Referência da Classe `src.domain.repositories.CsvExporter`

Diagrama de hierarquia da classe `src.domain.repositories.CsvExporter`:



Métodos Públicos

- bytes [export](#) (self, pd.DataFrame rows)

10.4.1 Descrição detalhada

```
@brief Protocolo para exportação de dados no formato CSV.
```

10.4.2 Documentação das funções

10.4.2.1 `export()`

```
bytes src.domain.repositories.CsvExporter.export (
    self,
    pd.DataFrame rows )
```

@brief Exporta um DataFrame para bytes no formato CSV.

A documentação para essa classe foi gerada a partir do seguinte arquivo:

- [domain/repositories.py](#)

10.5 Referência da Classe `src.domain.models.DiagnosticsInfo`

Atributos Públicos Estáticos

- list [odbc_drivers](#) [str]
- bool [connection_ok](#)
- str [connection_message](#)
- dict [active_context](#) [str, Any]
- datetime [checked_at](#)

10.5.1 Descrição detalhada

@brief Informações detalhadas de diagnóstico do sistema.

@param odbc_drivers Lista de drivers ODBC instalados no ambiente.
@param connection_ok Indica se a conexão com o banco está ativa.
@param connection_message Mensagem descritiva do estado da conexão.
@param active_context Contexto ativo no momento da verificação.
@param checked_at Timestamp da verificação de diagnóstico.

10.5.2 Atributos

10.5.2.1 `active_context`

```
dict src.domain.models.DiagnosticsInfo.active_context [str, Any] [static]
```

10.5.2.2 `checked_at`

```
datetime src.domain.models.DiagnosticsInfo.checked_at [static]
```

10.5.2.3 `connection_message`

```
str src.domain.models.DiagnosticsInfo.connection_message [static]
```

10.5.2.4 `connection_ok`

```
bool src.domain.models.DiagnosticsInfo.connection_ok [static]
```

10.5.2.5 `odbc_drivers`

```
list src.domain.models.DiagnosticsInfo.odbc_drivers [str] [static]
```

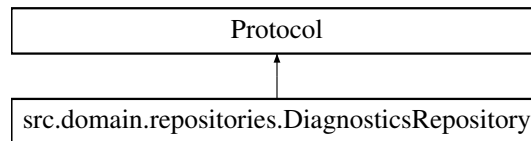
A documentação para essa classe foi gerada a partir do seguinte arquivo:

- [models.py](#)

10.6 Referência da Classe

src.domain.repositories.DiagnosticsRepository

Diagrama de hierarquia da classe src.domain.repositories.DiagnosticsRepository:



Métodos Públicos

- [DiagnosticsInfo diagnostics](#) (self, dict[str, object] active_context)

10.6.1 Descrição detalhada

@brief Protocolo para obtenção de informações de diagnóstico do sistema.

10.6.2 Documentação das funções

10.6.2.1 diagnostics()

```

DiagnosticsInfo src.domain.repositories.DiagnosticsRepository.diagnostics (
    self,
    dict[str, object] active_context )
  
```

@brief Retorna diagnóstico completo: drivers ODBC, estado da conexão e contexto ativo.

A documentação para essa classe foi gerada a partir do seguinte arquivo:

- [domain/repositories.py](#)

10.7 Referência da Classe src.application.services.DiagnosticsService

Métodos Públicos

- [__init__](#) (self, [DiagnosticsRepository repository](#))
- dict[str, object] [get](#) (self, dict[str, object] active_context)

Atributos Públicos

- [repository](#)

10.7.1 Descrição detalhada

@brief Serviço para obtenção de diagnósticos do sistema.

10.7.2 Construtores e Destrutores

10.7.2.1 __init__()

```

src.application.services.DiagnosticsService.__init__ (
    self,
    DiagnosticsRepository repository )
  
```

@brief Inicializa o serviço com repositório de diagnósticos.

@param repository Repositório de diagnósticos.

10.7.3 Documentação das funções

10.7.3.1 `get()`

```
dict[str, object] src.application.services.DiagnosticsService.get (
    self,
    dict[str, object] active_context )
```

@brief Obtém diagnóstico completo da conexão e ambiente.

@param `active_context` Contexto ativo para incluir no diagnóstico.
@return Dicionário com drivers ODBC, estado da conexão e contexto.

10.7.4 Atributos

10.7.4.1 `repository`

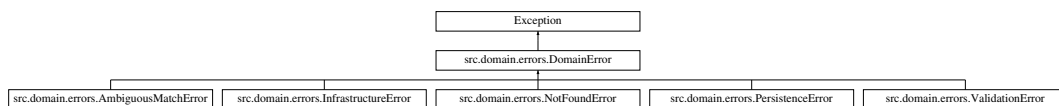
```
src.application.services.DiagnosticsService.repository
```

A documentação para essa classe foi gerada a partir do seguinte arquivo:

- [services.py](#)

10.8 Referência da Classe `src.domain.errors.DomainError`

Diagrama de hierarquia da classe `src.domain.errors.DomainError`:



10.8.1 Descrição detalhada

@brief Erro base da camada de domínio.

Todas as exceções de domínio herdam desta classe.

A documentação para essa classe foi gerada a partir do seguinte arquivo:

- [errors.py](#)

10.9 Referência da Classe `src.domain.models.DynamicDataResult`

Métodos Públicos

- `int total_rows` (self)

Atributos Públicos

- `rows`

Atributos Públicos Estáticos

- `pd rows` .DataFrame
- `str ordered_by`

10.9.1 Descrição detalhada

@brief Resultado da consulta de dados dinâmicos (série temporal).

@param `rows` DataFrame pandas com as linhas da tabela dinâmica.
@param `ordered_by` Nome da coluna utilizada para ordenação.

10.9.2 Documentação das funções

10.9.2.1 total_rows()

```
int src.domain.models.DynamicDataResult.total_rows (
    self )
@brief Retorna o número total de linhas carregadas.
```

10.9.3 Atributos

10.9.3.1 ordered_by

```
str src.domain.models.DynamicDataResult.ordered_by [static]
```

10.9.3.2 rows [1/2]

```
pd src.domain.models.DynamicDataResult.rows .DataFrame [static]
```

10.9.3.3 rows [2/2]

```
src.domain.models.DynamicDataResult.rows
```

A documentação para essa classe foi gerada a partir do seguinte arquivo:

- [models.py](#)

10.10 Referência da Classe src.application.dto.ExportPdfRequest

Atributos Públicos Estáticos

- str [export_mode](#)
- dict [metadata](#) [str, Any]
- bytes [graph_png](#) | None
- int [table_sample_max_rows](#) = 500

10.10.1 Descrição detalhada

```
@brief DTO com parâmetros para exportação de relatório PDF.

@param export_mode Modo de exportação ("config_atual" ou "todos_valores").
@param metadata Metadados do ensaio para incluir no relatório.
@param graph_png Imagem do gráfico em PNG, ou None.
@param table_sample_max_rows Máximo de linhas na amostra da tabela (padrão: 500).
```

10.10.2 Atributos

10.10.2.1 export_mode

```
str src.application.dto.ExportPdfRequest.export_mode [static]
```

10.10.2.2 graph_png

```
bytes src.application.dto.ExportPdfRequest.graph_png | None [static]
```

10.10.2.3 metadata

```
dict src.application.dto.ExportPdfRequest.metadata [str, Any] [static]
```

10.10.2.4 table_sample_max_rows

```
int src.application.dto.ExportPdfRequest.table_sample_max_rows = 500 [static]
```

A documentação para essa classe foi gerada a partir do seguinte arquivo:

- [dto.py](#)

10.11 Referência da Classe `src.application.services.ExportService`

Métodos Públicos

- `__init__` (self, `CsvExporter` `csv_exporter`, `PdfExporter` `pdf_exporter`)
- bytes `export_csv` (self, `pd.DataFrame` `rows`)
- bytes `export_pdf` (self, *`pd.DataFrame` `rows`, `pd.DataFrame` `static_row`, `ExportPdfRequest` `request`)

Atributos Públicos

- `csv_exporter`
- `pdf_exporter`

10.11.1 Descrição detalhada

@brief Serviço para exportação de dados em CSV e PDF.

10.11.2 Construtores e Destrutores

10.11.2.1 `__init__()`

```
src.application.services.ExportService.__init__ (
    self,
    CsvExporter csv_exporter,
    PdfExporter pdf_exporter )
```

@brief Inicializa o serviço com os exportadores.

@param `csv_exporter` Implementação do exportador CSV.

@param `pdf_exporter` Implementação do exportador PDF.

10.11.3 Documentação das funções

10.11.3.1 `export_csv()`

```
bytes src.application.services.ExportService.export_csv (
    self,
    pd.DataFrame rows )
```

@brief Exporta dados para CSV com separador ponto-e-vírgula.

@param `rows` `DataFrame` com os dados a exportar.

@return Bytes do arquivo CSV.

10.11.3.2 `export_pdf()`

```
bytes src.application.services.ExportService.export_pdf (
    self,
    *pd.DataFrame rows,
    pd.DataFrame static_row,
    ExportPdfRequest request )
```

@brief Gera relatório PDF com gráfico e dados do ensaio.

No modo "config_atual", limita a amostra da tabela ao máximo configurado.

@param `rows` Dados dinâmicos da série temporal.

@param `static_row` Dados estáticos de configuração.

@param `request` Parâmetros de exportação.

@return Bytes do arquivo PDF.

10.11.4 Atributos

10.11.4.1 csv_exporter

`src.application.services.ExportService.csv_exporter`

10.11.4.2 pdf_exporter

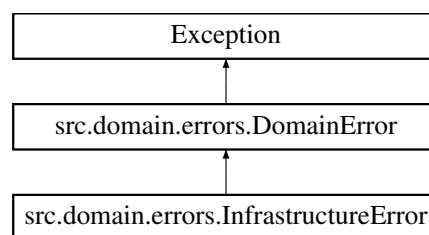
`src.application.services.ExportService.pdf_exporter`

A documentação para essa classe foi gerada a partir do seguinte arquivo:

- [services.py](#)

10.12 Referência da Classe `src.domain.errors.InfrastructureError`

Diagrama de hierarquia da classe `src.domain.errors.InfrastructureError`:



10.12.1 Descrição detalhada

@brief Erro disparado quando ocorre uma falha de infraestrutura, como conexão SQL.

A documentação para essa classe foi gerada a partir do seguinte arquivo:

- [errors.py](#)

10.13 Referência da Classe `src.application.dto.LoadTestResponse`

Atributos Públicos Estáticos

- Any [rows](#)
- Any [static_row](#)
- str [force_column](#) | None
- list [available_y_columns](#) [str]
- dict [meta](#) [str, Any]

10.13.1 Descrição detalhada

@brief DTO com a resposta completa do carregamento de um ensaio.

@param rows DataFrame com os dados dinâmicos da série temporal.

@param static_row DataFrame com os dados estáticos de configuração.

@param force_column Nome da coluna de força detectada, ou None.

@param available_y_columns Lista de colunas Y disponíveis para plotagem.

@param meta Metadados da consulta (ordenação, total de linhas, tabela usada).

10.13.2 Atributos

10.13.2.1 available_y_columns

list `src.application.dto.LoadTestResponse.available_y_columns` [str] [static]

10.13.2.2 force_column

```
str src.application.dto.LoadTestResponse.force_column | None [static]
```

10.13.2.3 meta

```
dict src.application.dto.LoadTestResponse.meta [str, Any] [static]
```

10.13.2.4 rows

```
Any src.application.dto.LoadTestResponse.rows [static]
```

10.13.2.5 static_row

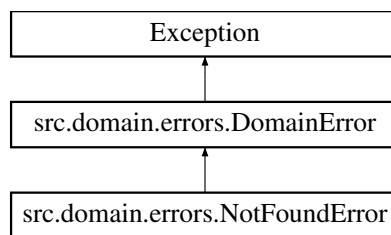
```
Any src.application.dto.LoadTestResponse.static_row [static]
```

A documentação para essa classe foi gerada a partir do seguinte arquivo:

- [dto.py](#)

10.14 Referência da Classe src.domain.errors.NotFoundError

Diagrama de hierarquia da classe src.domain.errors.NotFoundError:

**10.14.1 Descrição detalhada**

```
@brief Erro disparado quando um recurso esperado não é encontrado.
```

A documentação para essa classe foi gerada a partir do seguinte arquivo:

- [errors.py](#)

10.15 Referência da Classe src.domain.models.ObservationCommand**Atributos Públicos Estáticos**

- int [cyl_num](#)
- str [test_id](#)
- Any [local_col](#)
- str [y_column_name](#)
- float [y_value](#) | None
- str [obs_text](#) | None

10.15.1 Descrição detalhada

```
@brief Comando para criar ou atualizar uma observação em uma coordenada.
```

Veicula todos os dados necessários para localizar um ponto na série temporal e associar-lhe um texto descritivo de observação.

```
@param cyl_num Número do cilindro.
```

```
@param test_id Identificador do ensaio.
```

```
@param local_col Instante (coordenada X) da observação.
@param y_column_name Nome da variável Y alvo.
@param y_value Valor numérico da variável Y no ponto.
@param obs_text Texto livre da observação (pode ser None).
```

10.15.2 Atributos

10.15.2.1 cyl_num

```
int src.domain.models.ObservationCommand.cyl_num [static]
```

10.15.2.2 local_col

```
Any src.domain.models.ObservationCommand.local_col [static]
```

10.15.2.3 obs_text

```
str src.domain.models.ObservationCommand.obs_text | None [static]
```

10.15.2.4 test_id

```
str src.domain.models.ObservationCommand.test_id [static]
```

10.15.2.5 y_column_name

```
str src.domain.models.ObservationCommand.y_column_name [static]
```

10.15.2.6 y_value

```
float src.domain.models.ObservationCommand.y_value | None [static]
```

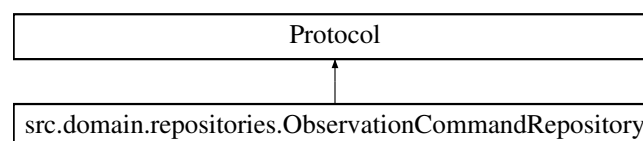
A documentação para essa classe foi gerada a partir do seguinte arquivo:

- [models.py](#)

10.16 Referência da Classe

src.domain.repositories.ObservationCommandRepository

Diagrama de hierarquia da classe src.domain.repositories.ObservationCommandRepository:



Métodos Públicos

- [ObservationResult upsert_observation](#) (self, [ObservationCommand](#) cmd)
- [ObservationRecord|None get_observation](#) (self, [ObservationLookup](#) lookup)
- bool [delete_observation](#) (self, [ObservationLookup](#) lookup)

10.16.1 Descrição detalhada

```
@brief Protocolo para operações CRUD de observações por coordenada.
```

10.16.2 Documentação das funções

10.16.2.1 `delete_observation()`

```
bool src.domain.repositories.ObservationCommandRepository.delete_observation (
    self,
    ObservationLookup lookup )
```

@brief Remove uma observação pela chave composta.

10.16.2.2 `get_observation()`

```
ObservationRecord | None src.domain.repositories.ObservationCommandRepository.get_observation (
    self,
    ObservationLookup lookup )
```

@brief Recupera uma observação existente pela chave composta.

10.16.2.3 `upsert_observation()`

```
ObservationResult src.domain.repositories.ObservationCommandRepository.upsert_observation (
    self,
    ObservationCommand cmd )
```

@brief Cria ou atualiza uma observação em uma coordenada específica.

A documentação para essa classe foi gerada a partir do seguinte arquivo:

- [domain/repositories.py](#)

10.17 Referência da Classe `src.domain.models.ObservationLookup`

Atributos Públicos Estáticos

- int `cyl_num`
- str `test_id`
- Any `local_col`
- str `y_column_name`
- float `y_value` | None

10.17.1 Descrição detalhada

@brief Chave de busca para recuperar uma observação existente.

```
@param cyl_num Número do cilindro.
@param test_id Identificador do ensaio.
@param local_col Instante (coordenada X) da observação.
@param y_column_name Nome da variável Y alvo.
@param y_value Valor numérico da variável Y no ponto.
```

10.17.2 Atributos

10.17.2.1 `cyl_num`

```
int src.domain.models.ObservationLookup.cyl_num [static]
```

10.17.2.2 `local_col`

```
Any src.domain.models.ObservationLookup.local_col [static]
```

10.17.2.3 test_id

```
str src.domain.models.ObservationLookup.test_id [static]
```

10.17.2.4 y_column_name

```
str src.domain.models.ObservationLookup.y_column_name [static]
```

10.17.2.5 y_value

```
float src.domain.models.ObservationLookup.y_value | None [static]
```

A documentação para essa classe foi gerada a partir do seguinte arquivo:

- [models.py](#)

10.18 Referência da Classe src.domain.models.ObservationRecord

Atributos Públicos Estáticos

- str [obs_text](#) | None
- str [y_column_name](#) | None
- float [val_obs](#) | None

10.18.1 Descrição detalhada

@brief Registro de observação recuperado do banco de dados.

@param obs_text Texto descritivo da observação (pode ser None).

@param y_column_name Nome da variável Y associada (pode ser None).

@param val_obs Valor numérico observado (pode ser None).

10.18.2 Atributos

10.18.2.1 obs_text

```
str src.domain.models.ObservationRecord.obs_text | None [static]
```

10.18.2.2 val_obs

```
float src.domain.models.ObservationRecord.val_obs | None [static]
```

10.18.2.3 y_column_name

```
str src.domain.models.ObservationRecord.y_column_name | None [static]
```

A documentação para essa classe foi gerada a partir do seguinte arquivo:

- [models.py](#)

10.19 Referência da Classe src.domain.models.ObservationResult

Atributos Públicos Estáticos

- bool [updated](#)
- int [matched_rows](#)
- dict [columns_used](#) [str, str]

10.19.1 Descrição detalhada

@brief Resultado da operação de upsert de observação.

@param updated True se houve atualização de registro existente.

@param matched_rows Quantidade de linhas afetadas pela operação.

@param columns_used Mapeamento das colunas utilizadas na operação.

10.19.2 Atributos

10.19.2.1 columns_used

```
dict src.domain.models.ObservationResult.columns_used [str, str] [static]
```

10.19.2.2 matched_rows

```
int src.domain.models.ObservationResult.matched_rows [static]
```

10.19.2.3 updated

```
bool src.domain.models.ObservationResult.updated [static]
```

A documentação para essa classe foi gerada a partir do seguinte arquivo:

- [models.py](#)

10.20 Referência da Classe src.application.services.ObservationService

Métodos Públicos

- `__init__` (self, [ObservationCommandRepository repository](#))
- dict[str, object] `upsert` (self, [ObservationCommand cmd](#))
- dict[str, object]|None `get` (self, [ObservationLookup lookup](#))
- bool `delete` (self, [ObservationLookup lookup](#))

Atributos Públicos

- [repository](#)

10.20.1 Descrição detalhada

@brief Serviço para operações CRUD de observações por coordenada.

10.20.2 Construtores e Destrutores

10.20.2.1 __init__()

```
src.application.services.ObservationService.__init__ (
    self,
    ObservationCommandRepository repository )
```

@brief Inicializa o serviço com repositório de observações.

@param repository Repositório de persistência de observações.

10.20.3 Documentação das funções

10.20.3.1 delete()

```
bool src.application.services.ObservationService.delete (
    self,
    ObservationLookup lookup )
```

@brief Remove uma observação pela chave composta.

@param lookup Chave de busca da observação a remover.

@return True se excluída, False se não encontrada.

10.20.3.2 get()

```
dict[str, object] | None src.application.services.ObservationService.get (
    self,
    ObservationLookup lookup )

@brief Recupera uma observação pela chave composta.

@param lookup Chave de busca (cilindro, teste, coordenada, variável).
@return Dicionário com dados da observação ou None se não encontrada.
```

10.20.3.3 upsert()

```
dict[str, object] src.application.services.ObservationService.upsert (
    self,
    ObservationCommand cmd )

@brief Cria ou atualiza uma observação em uma coordenada.

@param cmd Comando com dados da observação.
@return Dicionário com o resultado da operação.
```

10.20.4 Atributos

10.20.4.1 repository

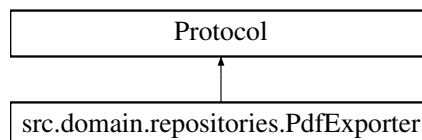
```
src.application.services.ObservationService.repository
```

A documentação para essa classe foi gerada a partir do seguinte arquivo:

- [services.py](#)

10.21 Referência da Classe src.domain.repositories.PdfExporter

Diagrama de hierarquia da classe src.domain.repositories.PdfExporter:



Métodos Públicos

- bytes [export](#) (self, *pd.DataFrame rows, pd.DataFrame static_row, dict[str, object] metadata, bytes|None graph_png, int sample_max_rows)

10.21.1 Descrição detalhada

```
@brief Protocolo para exportação de relatório técnico em PDF.
```

10.21.2 Documentação das funções

10.21.2.1 export()

```
bytes src.domain.repositories.PdfExporter.export (
    self,
    *pd.DataFrame rows,
    pd.DataFrame static_row,
    dict[str, object] metadata,
    bytes | None graph_png,
    int sample_max_rows )
```

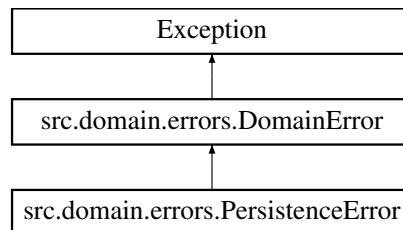
@brief Gera um relatório PDF com gráfico, tabela estática e amostra dos dados dinâmicos.

A documentação para essa classe foi gerada a partir do seguinte arquivo:

- [domain/repositories.py](#)

10.22 Referência da Classe `src.domain.errors.PersistenceError`

Diagrama de hierarquia da classe `src.domain.errors.PersistenceError`:



10.22.1 Descrição detalhada

@brief Erro disparado quando uma operação de persistência (insert/update/delete) falha.

A documentação para essa classe foi gerada a partir do seguinte arquivo:

- [errors.py](#)

10.23 Referência da Classe `src.infrastructure.cache_redis.RedisCache`

Métodos Públicos

- `__init__` (self, int|None ttl_seconds=None)
- bool `available` (self)
- Any|None `get` (self, str key)
- None `set` (self, str key, Any value)
- None `delete` (self, str key)
- None `flush` (self)
- str `__repr__` (self)

Atributos Públicos

- `ttl`

Membros protegidos

- Any|None `_fallback_get` (self, str key)
- None `_fallback_set` (self, str key, Any value)

Atributos Protegidos

- `_connected`
- `_client`

10.23.1 Descrição detalhada

@brief Cache Redis com fallback transparente em memória.

Interface compatível com `TTLCache`. Tenta conectar ao Redis na inicialização; se falhar, opera em modo fallback com dicionário em memória.

10.23.2 Construtores e Destrutores

10.23.2.1 `__init__()`

```
src.infrastructure.cache_redis.RedisCache.__init__ (
    self,
    int | None ttl_seconds = None )

@brief Inicializa o cache Redis com fallback.

@param ttl_seconds TTL em segundos (padrão: PIFF_CACHE_TTL_SECONDS ou 300).
```

10.23.3 Documentação das funções

10.23.3.1 `__repr__()`

```
str src.infrastructure.cache_redis.RedisCache.__repr__ (
    self )

@brief Representação textual indicando o modo ativo.
```

10.23.3.2 `_fallback_get()`

```
Any | None src.infrastructure.cache_redis.RedisCache._fallback_get (
    self,
    str key ) [protected]

@brief Recupera do fallback em memória.
```

10.23.3.3 `_fallback_set()`

```
None src.infrastructure.cache_redis.RedisCache._fallback_set (
    self,
    str key,
    Any value ) [protected]

@brief Armazena no fallback em memória.
```

10.23.3.4 `available()`

```
bool src.infrastructure.cache_redis.RedisCache.available (
    self )

@brief Indica se a conexão com o Redis está ativa.
```

10.23.3.5 `delete()`

```
None src.infrastructure.cache_redis.RedisCache.delete (
    self,
    str key )

@brief Remove uma entrada do cache.

@param key Chave da entrada a remover.
```

10.23.3.6 `flush()`

```
None src.infrastructure.cache_redis.RedisCache.flush (
    self )

@brief Limpa todo o cache (Redis e fallback).

@note Use com cuidado -- afeta todas as chaves no banco Redis.
```

10.23.3.7 `get()`

```
Any | None src.infrastructure.cache_redis.RedisCache.get (
    self,
    str key )
```

@brief Recupera um valor do cache (Redis ou fallback).

@param key Chave da entrada.

@return Valor desserializado ou None.

10.23.3.8 `set()`

```
None src.infrastructure.cache_redis.RedisCache.set (
    self,
    str key,
    Any value )
```

@brief Armazena um valor no cache com TTL.

@param key Chave da entrada.

@param value Valor a armazenar (serializado para JSON).

10.23.4 Atributos

10.23.4.1 `_client`

```
src.infrastructure.cache_redis.RedisCache._client [protected]
```

10.23.4.2 `_connected`

```
src.infrastructure.cache_redis.RedisCache._connected [protected]
```

10.23.4.3 `ttl`

```
src.infrastructure.cache_redis.RedisCache.ttl
```

A documentação para essa classe foi gerada a partir do seguinte arquivo:

- [cache_redis.py](#)

10.24 Referência da Classe

`src.infrastructure.exporters.csv_exporter.SemicolonCsvExporter`

Métodos Públicos

- bytes [export](#) (self, pd.DataFrame rows)

10.24.1 Descrição detalhada

@brief Exporta DataFrames para CSV com separador ponto-e-vírgula.

Utiliza encoding UTF-8 com BOM (utf-8-sig) para compatibilidade com Microsoft Excel em locale português brasileiro.

10.24.2 Documentação das funções

10.24.2.1 `export()`

```
bytes src.infrastructure.exporters.csv_exporter.SemicolonCsvExporter.export (
    self,
    pd.DataFrame rows )
```

```
@brief Converte um DataFrame em bytes CSV.

@param rows DataFrame com os dados a exportar.
@return Conteúdo do arquivo CSV em bytes.
```

A documentação para essa classe foi gerada a partir do seguinte arquivo:

- [csv_exporter.py](#)

10.25 Referência da Classe `src.config.settings.Settings`

Métodos Públicos

- `tuple[int,...] allowed_cylinders (self)`

Atributos Públicos Estáticos

- `str sql_server = os.getenv("PIFF_SQL_SERVER", "localhost")`
- `str sql_database = os.getenv("PIFF_SQL_DATABASE", "Projeto_54")`
- `str sql_username = os.getenv("PIFF_SQL_USERNAME", "sa")`
- `str sql_password = os.getenv("PIFF_SQL_PASSWORD", "P@ssw0rd_Projeto54!")`
- `str sql_driver = os.getenv("PIFF_SQL_DRIVER", "ODBC Driver 18 for SQL Server")`
- `bool trusted_connection = _as_bool(os.getenv("PIFF_SQL_TRUSTED_CONNECTION"), default=False)`
- `int cache_ttl_seconds = int(os.getenv("PIFF_CACHE_TTL_SECONDS", "300"))`

10.25.1 Descrição detalhada

```
@brief Configurações imutáveis do sistema.
```

Lê valores de variáveis de ambiente com fallback para padrões que funcionam em ambiente de desenvolvimento local.

```
@param sql_server Host do SQL Server (PIFF_SQL_SERVER).
@param sql_database Nome do banco de dados (PIFF_SQL_DATABASE).
@param sql_username Usuário SQL (PIFF_SQL_USERNAME).
@param sql_password Senha SQL (PIFF_SQL_PASSWORD).
@param sql_driver Driver ODBC (PIFF_SQL_DRIVER).
@param trusted_connection Autenticação Windows (PIFF_SQL_TRUSTED_CONNECTION).
@param cache_ttl_seconds TTL do cache em segundos (PIFF_CACHE_TTL_SECONDS).
```

10.25.2 Documentação das funções

10.25.2.1 `allowed_cylinders()`

```
tuple[int, ...] src.config.settings.Settings.allowed_cylinders (
    self )
```

```
@brief Tupla com os números de cilindros suportados (1 a 6).
```

10.25.3 Atributos

10.25.3.1 `cache_ttl_seconds`

```
int src.config.settings.Settings.cache_ttl_seconds = int(os.getenv("PIFF_CACHE_TTL_SECONDS",
"300")) [static]
```

10.25.3.2 `sql_database`

```
str src.config.settings.Settings.sql_database = os.getenv("PIFF_SQL_DATABASE", "Projeto_54")
[static]
```

10.25.3.3 `sql_driver`

```
str src.config.settings.Settings.sql_driver = os.getenv("PIFF_SQL_DRIVER", "ODBC Driver 18 for SQL Server") [static]
```

10.25.3.4 `sql_password`

```
str src.config.settings.Settings.sql_password = os.getenv("PIFF_SQL_PASSWORD", "P@ssw0rd_↵Projeto54!") [static]
```

10.25.3.5 `sql_server`

```
str src.config.settings.Settings.sql_server = os.getenv("PIFF_SQL_SERVER", "localhost") [static]
```

10.25.3.6 `sql_username`

```
str src.config.settings.Settings.sql_username = os.getenv("PIFF_SQL_USERNAME", "sa") [static]
```

10.25.3.7 `trusted_connection`

```
bool src.config.settings.Settings.trusted_connection = _as_bool(os.getenv("PIFF_SQL_TRUSTED_↵CONNECTION"), default=False) [static]
```

A documentação para essa classe foi gerada a partir do seguinte arquivo:

- [settings.py](#)

10.26 Referência da Classe `src.infrastructure.sql.repositories.SqlDiagnosticsRepository`

Métodos Públicos

- `__init__` (self, [SqlServerConnection](#) conn)
- [DiagnosticsInfo](#) diagnostics (self, dict[str, object] active_context)

Atributos Públicos

- [conn](#)

10.26.1 Descrição detalhada

@brief Repositório SQL para diagnóstico do sistema.

Verifica drivers ODBC instalados, testa a conexão com o banco e retorna o contexto ativo para depuração.

10.26.2 Construtores e Destrutores

10.26.2.1 `__init__()`

```
src.infrastructure.sql.repositories.SqlDiagnosticsRepository.__init__ (
    self,
    SqlServerConnection conn )
```

@brief Inicializa o repositório.

@param conn Gerenciador de conexão ODBC.

10.26.3 Documentação das funções

10.26.3.1 diagnostics()

```

DiagnosticsInfo src.infrastructure.sql.repositories.SqlDiagnosticsRepository.diagnostics (
    self,
    dict[str, object] active_context )

```

@brief Obtém diagnóstico completo do sistema.

@param active_context Contexto ativo (cilindro, teste, linhas carregadas).

@return DiagnosticsInfo com drivers, status da conexão e contexto.

10.26.4 Atributos

10.26.4.1 conn

```
src.infrastructure.sql.repositories.SqlDiagnosticsRepository.conn
```

A documentação para essa classe foi gerada a partir do seguinte arquivo:

- [infrastructure/sql/repositories.py](#)

10.27 Referência da Classe

src.infrastructure.sql.repositories.SqlObservationRepository

Métodos Públicos

- [__init__](#) (self, [SqlServerConnection](#) conn)
- [ObservationResult](#) [upsert_observation](#) (self, [ObservationCommand](#) cmd)
- [ObservationRecord](#)|None [get_observation](#) (self, [ObservationLookup](#) lookup)
- bool [delete_observation](#) (self, [ObservationLookup](#) lookup)

Atributos Públicos

- [conn](#)

Membros protegidos estáticos

- tuple[str, str, str] [_match_query](#) (int cyl_num, str y_column_name, str include_select)

10.27.1 Descrição detalhada

@brief Repositório SQL para operações CRUD de observações.

Implementa ObservationCommandRepository utilizando UPDATE direto nas colunas OBS, val_obs e y_column_name da tabela dinâmica do cilindro.

10.27.2 Construtores e Destrutores

10.27.2.1 __init__()

```

src.infrastructure.sql.repositories.SqlObservationRepository.__init__ (
    self,
    SqlServerConnection conn )

```

@brief Inicializa o repositório.

@param conn Gerenciador de conexão ODBC.

10.27.3 Documentação das funções

10.27.3.1 _match_query()

```
tuple[str, str, str] src.infrastructure.sql.repositories.SqlObservationRepository._match_query
(
    int cyl_num,
    str y_column_name,
    str include_select ) [static], [protected]
```

@brief Constrói consulta de matching por tolerância numérica.

@param cyl_num Número do cilindro.

@param y_column_name Nome da coluna Y.

@param include_select Cláusula SELECT para a consulta.

@return Tupla (query, obs_col, val_col_y_name).

10.27.3.2 delete_observation()

```
bool src.infrastructure.sql.repositories.SqlObservationRepository.delete_observation (
    self,
    ObservationLookup lookup )
```

@brief Remove (limpa) a observação de uma coordenada.

Seta as colunas OBS, val_obs e y_column_name para NULL.

@param lookup Chave de busca da observação a remover.

@return True se uma linha foi alterada.

@raise AmbiguousMatchError Se mais de uma linha for afetada.

10.27.3.3 get_observation()

```
ObservationRecord | None src.infrastructure.sql.repositories.SqlObservationRepository.get_↵
observation (
    self,
    ObservationLookup lookup )
```

@brief Recupera uma observação pela chave composta.

@param lookup Chave de busca.

@return ObservationRecord ou None se não encontrada.

@raise AmbiguousMatchError Se mais de um registro for encontrado.

10.27.3.4 upsert_observation()

```
ObservationResult src.infrastructure.sql.repositories.SqlObservationRepository.upsert_observation
(
    self,
    ObservationCommand cmd )
```

@brief Cria ou atualiza uma observação em uma coordenada.

Busca a linha exata por ID_Testes, LocalCol e tolerância numérica.
Se encontrar exatamente 1 linha, faz UPDATE nas colunas de observação.

@param cmd Comando com dados da observação.

@return ObservationResult com matched_rows e colunas utilizadas.

@raise NotFoundError Se nenhuma linha corresponder aos critérios.

@raise AmbiguousMatchError Se mais de uma linha corresponder.

@raise PersistenceError Se o UPDATE não afetar exatamente 1 linha.

10.27.4 Atributos

10.27.4.1 conn

`src.infrastructure.sql.repositories.SqlObservationRepository.conn`

A documentação para essa classe foi gerada a partir do seguinte arquivo:

- [infrastructure/sql/repositories.py](#)

10.28 Referência da Classe

`src.infrastructure.sql.connection.SqlServerConnection`

Métodos Públicos

- `__init__` (self, [Settings settings](#))
- str `build_connection_string` (self)
- `connect` (self)
- `connect_shared` (self)
- tuple[list[str], list[tuple[Any,...]]] `fetch_all` (self, str query, Iterable[Any] params=())
- tuple[list[str], list[tuple[Any,...]]] `fetch_all_on` (self, pyodbc.Connection conn, str query, Iterable[Any] params=())
- int `execute` (self, str query, Iterable[Any] params=())
- list[int] `execute_transaction` (self, list[tuple[str, tuple[Any,...]]] statements)
- list[str] `list_drivers` (self)

Atributos Públicos

- [settings](#)

10.28.1 Descrição detalhada

@brief Gerencia conexões com o SQL Server via ODBC.

Suporta autenticação SQL (usuário/senha) e trusted connection (Windows).
Oferece conexões compartilhadas para otimizar múltiplas consultas.

10.28.2 Construtores e Destrutores

10.28.2.1 `__init__()`

```
src.infrastructure.sql.connection.SqlServerConnection.__init__ (
    self,
    Settings settings )
```

@brief Inicializa a conexão com as configurações fornecidas.

@param settings Configurações do sistema (servidor, banco, credenciais).

10.28.3 Documentação das funções

10.28.3.1 `build_connection_string()`

```
str src.infrastructure.sql.connection.SqlServerConnection.build_connection_string (
    self )
```

@brief Constrói a string de conexão ODBC.

Suporta trusted connection (autenticação Windows) ou SQL Auth.

@return String formatada para `pyodbc.connect()`.

10.28.3.2 `connect()`

```
src.infrastructure.sql.connection.SqlServerConnection.connect (
    self )

@brief Abre uma nova conexão com o SQL Server (context manager).

@yield Conexão pyodbc ativa.
@raise InfrastructureError Se a conexão falhar.
```

10.28.3.3 `connect_shared()`

```
src.infrastructure.sql.connection.SqlServerConnection.connect_shared (
    self )

@brief Abre uma conexão compartilhada para múltiplas operações.

Todas as chamadas fetch_all_on / execute_on reutilizam esta conexão,
evitando o overhead de ~300ms do handshake por consulta.

@yield Conexão pyodbc ativa.
@raise InfrastructureError Se a conexão falhar.
```

10.28.3.4 `execute()`

```
int src.infrastructure.sql.connection.SqlServerConnection.execute (
    self,
    str query,
    Iterable[Any] params = () )

@brief Executa um comando SQL (INSERT/UPDATE/DELETE).

@param query String SQL.
@param params Parâmetros posicionais.
@return Número de linhas afetadas.
```

10.28.3.5 `execute_transaction()`

```
list[int] src.infrastructure.sql.connection.SqlServerConnection.execute_transaction (
    self,
    list[tuple[str, tuple[Any, ...]]] statements )

@brief Executa múltiplos comandos em uma transação atômica.

Commit se todos bem-sucedidos, rollback em caso de erro.

@param statements Lista de tuplas (query, params).
@return Lista com contagem de linhas afetadas por comando.
@raise InfrastructureError Se qualquer comando falhar.
```

10.28.3.6 `fetch_all()`

```
tuple[list[str], list[tuple[Any, ...]]] src.infrastructure.sql.connection.SqlServerConnection.↵
fetch_all (
    self,
    str query,
    Iterable[Any] params = () )

@brief Executa uma consulta SQL em uma nova conexão.

@param query String SQL.
@param params Parâmetros posicionais.
@return Tupla (colunas, linhas).
```

10.28.3.7 fetch_all_on()

```
tuple[list[str], list[tuple[Any, ...]]] src.infrastructure.sql.connection.SqlServerConnection.fetch_all_on (
    self,
    pyodbc.Connection conn,
    str query,
    Iterable[Any] params = () )

@brief Executa uma consulta em uma conexão já aberta.

@param conn Conexão pyodbc ativa (de connect_shared).
@param query String SQL.
@param params Parâmetros posicionais.
@return Tupla (colunas, linhas).
```

10.28.3.8 list_drivers()

```
list[str] src.infrastructure.sql.connection.SqlServerConnection.list_drivers (
    self )

@brief Lista todos os drivers ODBC instalados no sistema.

@return Lista de nomes de drivers ODBC disponíveis.
```

10.28.4 Atributos

10.28.4.1 settings

src.infrastructure.sql.connection.SqlServerConnection.settings
A documentação para essa classe foi gerada a partir do seguinte arquivo:

- [connection.py](#)

10.29 Referência da Classe

src.infrastructure.sql.repositories.SqlTestQueryRepository

Métodos Públicos

- [__init__](#) (self, [SqlServerConnection](#) conn)
- [list_test_ids](#) (self, int cyl_num)
- [DynamicDataResult load_dynamic_rows](#) (self, [TestContext](#) ctx, list[str]|None selected_columns=None, Any _conn=None)
- [StaticDataResult load_static_row](#) (self, [TestContext](#) ctx, Any _conn=None)
- [str|None detect_force_column](#) (self, int cyl_num, Any _conn=None)
- [list\[str\] load_available_y_columns](#) (self, int cyl_num, Any _conn=None)

Atributos Públicos

- [conn](#)

Membros protegidos

- [_fetch](#) (self, str query, tuple params=(), Any _conn=None)
- [bool _table_exists](#) (self, str table_name, Any _conn=None)
- [list\[str\] _get_columns](#) (self, str table_name, Any _conn=None)
- [str|None _find_static_table](#) (self, int cyl_num, Any _conn=None)

10.29.1 Descrição detalhada

@brief Repositório SQL para consulta de dados de ensaio.

Implementa `TestQueryRepository` usando SQL Server. Mantém cache interno de colunas e tabelas descobertas durante o ciclo de vida da instância.

10.29.2 Construtores e Destrutores

10.29.2.1 `__init__()`

```
src.infrastructure.sql.repositories.SqlTestQueryRepository.__init__ (
    self,
    SqlServerConnection conn )
```

@brief Inicializa o repositório com uma conexão SQL Server.

@param conn Gerenciador de conexão ODBC.

10.29.3 Documentação das funções

10.29.3.1 `_fetch()`

```
src.infrastructure.sql.repositories.SqlTestQueryRepository._fetch (
    self,
    str query,
    tuple params = (),
    Any _conn = None ) [protected]
```

@brief Executa uma consulta, reutilizando conexão compartilhada se disponível.

@param query String SQL.

@param params Parâmetros posicionais.

@param _conn Conexão compartilhada opcional.

@return Tupla (colunas, linhas).

10.29.3.2 `_find_static_table()`

```
str | None src.infrastructure.sql.repositories.SqlTestQueryRepository._find_static_table (
    self,
    int cyl_num,
    Any _conn = None ) [protected]
```

@brief Descobre a tabela estática para um cilindro em uma única consulta.

Tenta variações do nome (com/sem acento) para compatibilidade.

@param cyl_num Número do cilindro.

@param _conn Conexão compartilhada opcional.

@return Nome da tabela encontrada ou None.

10.29.3.3 `_get_columns()`

```
list[str] src.infrastructure.sql.repositories.SqlTestQueryRepository._get_columns (
    self,
    str table_name,
    Any _conn = None ) [protected]
```

@brief Obtém a lista de colunas de uma tabela, com cache.

Usa `SELECT TOP 0` para obter metadados instantaneamente, evitando a consulta lenta `INFORMATION_SCHEMA.COLUMNS`.

@param table_name Nome da tabela.

@param _conn Conexão compartilhada opcional.

@return Lista de nomes de colunas.

10.29.3.4 `_table_exists()`

```
bool src.infrastructure.sql.repositories.SqlTestQueryRepository._table_exists (
    self,
    str table_name,
    Any _conn = None ) [protected]
```

@brief Verifica se uma tabela existe no banco, com cache.

@param table_name Nome da tabela.
@param _conn Conexão compartilhada opcional.
@return True se a tabela existir.

10.29.3.5 `detect_force_column()`

```
str | None src.infrastructure.sql.repositories.SqlTestQueryRepository.detect_force_column (
    self,
    int cyl_num,
    Any _conn = None )
```

@brief Detecta automaticamente a coluna de força na tabela dinâmica.

@param cyl_num Número do cilindro.
@param _conn Conexão compartilhada opcional.
@return Nome da coluna de força encontrada ou None.

10.29.3.6 `list_test_ids()`

```
list[str] src.infrastructure.sql.repositories.SqlTestQueryRepository.list_test_ids (
    self,
    int cyl_num )
```

@brief Lista IDs de teste distintos para um cilindro.

@param cyl_num Número do cilindro.
@return Lista de identificadores de ensaio.

10.29.3.7 `load_available_y_columns()`

```
list[str] src.infrastructure.sql.repositories.SqlTestQueryRepository.load_available_y_columns
(
    self,
    int cyl_num,
    Any _conn = None )
```

@brief Lista colunas numéricas disponíveis como variáveis Y.

@param cyl_num Número do cilindro.
@param _conn Conexão compartilhada opcional.
@return Lista de nomes de colunas Y disponíveis.

10.29.3.8 `load_dynamic_rows()`

```
DynamicDataResult src.infrastructure.sql.repositories.SqlTestQueryRepository.load_dynamic_rows
(
    self,
    TestContext ctx,
    list[str] | None selected_columns = None,
    Any _conn = None )
```

@brief Carrega os dados dinâmicos (série temporal) de um ensaio.

Usa SELECT com colunas explícitas alinhadas ao índice composto, evitando SELECT * que causava key lookups e full table scan de ~12s.

@param ctx Contexto do ensaio (cilindro + test_id).
 @param selected_columns Lista opcional de colunas Y para filtrar.
 @param _conn Conexão compartilhada opcional.
 @return DynamicDataResult com DataFrame ordenado por LocalCol.

10.29.3.9 load_static_row()

```
StaticDataResult src.infrastructure.sql.repositories.SqlTestQueryRepository.load_static_row (
    self,
    TestContext ctx,
    Any _conn = None )
```

@brief Carrega a linha de dados estáticos (configuração) de um ensaio.

@param ctx Contexto do ensaio.
 @param _conn Conexão compartilhada opcional.
 @return StaticDataResult com DataFrame e nome da tabela utilizada.

10.29.4 Atributos

10.29.4.1 conn

`src.infrastructure.sql.repositories.SqlTestQueryRepository.conn`

A documentação para essa classe foi gerada a partir do seguinte arquivo:

- [infrastructure/sql/repositories.py](#)

10.30 Referência da Classe `src.domain.models.StaticDataResult`

Atributos Públicos Estáticos

- `pd row` DataFrame
- `str table_used`

10.30.1 Descrição detalhada

@brief Resultado da consulta de dados estáticos (configuração do ensaio).

@param row DataFrame pandas com uma única linha de dados estáticos.
 @param table_used Nome da tabela utilizada na consulta.

10.30.2 Atributos

10.30.2.1 row

`pd src.domain.models.StaticDataResult.row` DataFrame [static]

10.30.2.2 table_used

`str src.domain.models.StaticDataResult.table_used` [static]

A documentação para essa classe foi gerada a partir do seguinte arquivo:

- [models.py](#)

10.31 Referência da Classe

src.infrastructure.exporters.pdf_exporter.TechnicalPdfExporter

Métodos Públicos

- bytes [export](#) (self, *pd.DataFrame rows, pd.DataFrame static_row, dict[str, object] metadata, bytes|None graph_png, int sample_max_rows)

10.31.1 Descrição detalhada

@brief Gera relatório técnico em PDF com dados de ensaio.

O relatório inclui: título, metadados, gráfico da série temporal (PNG), tabela de configuração estática e amostra dos dados dinâmicos.

10.31.2 Documentação das funções

10.31.2.1 export()

```
bytes src.infrastructure.exporters.pdf_exporter.TechnicalPdfExporter.export (
    self,
    *pd.DataFrame rows,
    pd.DataFrame static_row,
    dict[str, object] metadata,
    bytes | None graph_png,
    int sample_max_rows )
```

@brief Gera o relatório PDF completo.

Compõe o documento com: título, metadados, gráfico PNG, tabela de dados estáticos e amostra dos dados dinâmicos.

@param rows Dados dinâmicos da série temporal.
 @param static_row Dados estáticos de configuração.
 @param metadata Metadados do ensaio (banco, cilindro, teste).
 @param graph_png Imagem do gráfico em PNG (opcional).
 @param sample_max_rows Máximo de linhas na amostra da tabela.
 @return Bytes do arquivo PDF gerado.

A documentação para essa classe foi gerada a partir do seguinte arquivo:

- [pdf_exporter.py](#)

10.32 Referência da Classe src.domain.models.TestContext

Atributos Públicos Estáticos

- int [cyl_num](#)
- str [test_id](#)

10.32.1 Descrição detalhada

@brief Contexto que identifica unicamente um ensaio em um cilindro.

@param cyl_num Número do cilindro hidráulico (1 a 6).
 @param test_id Identificador único do ensaio (ex: C01_TESTE_001).

10.32.2 Atributos

10.32.2.1 cyl_num

```
int src.domain.models.TestContext.cyl_num [static]
```

10.32.2.2 `test_id`

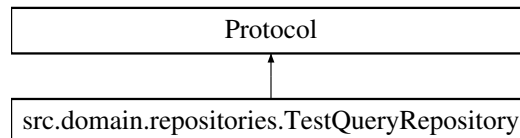
```
str src.domain.models.TestContext.test_id [static]
```

A documentação para essa classe foi gerada a partir do seguinte arquivo:

- [models.py](#)

10.33 Referência da Classe `src.domain.repositories.TestQueryRepository`

Diagrama de hierarquia da classe `src.domain.repositories.TestQueryRepository`:



Métodos Públicos

- `list[str] list_test_ids (self, int cyl_num)`
- `DynamicDataResult load_dynamic_rows (self, TestContext ctx, list[str]|None selected_columns=None)`
- `StaticDataResult load_static_row (self, TestContext ctx)`
- `str|None detect_force_column (self, int cyl_num)`
- `list[str] load_available_y_columns (self, int cyl_num)`

10.33.1 Descrição detalhada

@brief Protocolo para consulta de dados de ensaio no banco SQL.

Define as operações de leitura para tabelas dinâmicas (série temporal) e estáticas (configuração) por cilindro e ID de teste.

10.33.2 Documentação das funções

10.33.2.1 `detect_force_column()`

```
str | None src.domain.repositories.TestQueryRepository.detect_force_column (
    self,
    int cyl_num )
```

@brief Detecta automaticamente a coluna de força disponível na tabela dinâmica.

10.33.2.2 `list_test_ids()`

```
list[str] src.domain.repositories.TestQueryRepository.list_test_ids (
    self,
    int cyl_num )
```

@brief Lista os IDs de teste disponíveis para um cilindro.

10.33.2.3 `load_available_y_columns()`

```
list[str] src.domain.repositories.TestQueryRepository.load_available_y_columns (
    self,
    int cyl_num )
```

@brief Lista as colunas numéricas disponíveis como variáveis Y para plotagem.

10.33.2.4 load_dynamic_rows()

```
DynamicDataResult src.domain.repositories.TestQueryRepository.load_dynamic_rows (
    self,
    TestContext ctx,
    list[str] | None selected_columns = None )
```

@brief Carrega as linhas da tabela dinâmica (série temporal) para um ensaio.

10.33.2.5 load_static_row()

```
StaticDataResult src.domain.repositories.TestQueryRepository.load_static_row (
    self,
    TestContext ctx )
```

@brief Carrega a linha de configuração estática para um ensaio.

A documentação para essa classe foi gerada a partir do seguinte arquivo:

- [domain/repositories.py](#)

10.34 Referência da Classe src.application.services.TestReadService

Métodos Públicos

- `__init__` (self, [TestQueryRepository](#) repository, [TTLCache](#) cache)
- `list[str] list_test_ids` (self, int cyl_num)
- `LoadTestResponse load_test` (self, int cyl_num, str test_id, list[str]|None y_columns=None)

Atributos Públicos

- [repository](#)
- [cache](#)

Atributos Privados Estáticos

- bool `__test__` = False

10.34.1 Descrição detalhada

@brief Serviço para consulta e carregamento de dados de ensaio.

Orquestra a busca de dados dinâmicos (série temporal) e estáticos (configuração) com cache intermediário para reduzir consultas ao banco.

10.34.2 Construtores e Destrutores

10.34.2.1 __init__()

```
src.application.services.TestReadService.__init__ (
    self,
    TestQueryRepository repository,
    TTLCache cache )
```

@brief Inicializa o serviço com repositório e cache.

@param repository Repositório de consulta de dados de ensaio.
 @param cache Cache TTL para evitar consultas repetidas ao banco.

10.34.3 Documentação das funções

10.34.3.1 `list_test_ids()`

```
list[str] src.application.services.TestReadService.list_test_ids (
    self,
    int cyl_num )
```

@brief Lista os IDs de teste disponíveis para um cilindro, com cache.

@param `cyl_num` Número do cilindro.

@return Lista de identificadores de teste.

10.34.3.2 `load_test()`

```
LoadTestResponse src.application.services.TestReadService.load_test (
    self,
    int cyl_num,
    str test_id,
    list[str] | None y_columns = None )
```

@brief Carrega todos os dados de um ensaio (dinâmicos + estáticos) com cache.

Utiliza uma única conexão ODBC compartilhada para todas as consultas, evitando o overhead de ~300ms por handshake.

@param `cyl_num` Número do cilindro.

@param `test_id` Identificador do ensaio.

@param `y_columns` Lista opcional de colunas Y para filtrar.

@return `LoadTestResponse` com dados dinâmicos, estáticos e metadados.

@raise `NotFoundError` Se nenhum dado dinâmico for encontrado.

10.34.4 Atributos

10.34.4.1 `__test__`

```
bool src.application.services.TestReadService.__test__ = False [static], [private]
```

10.34.4.2 `cache`

```
src.application.services.TestReadService.cache
```

10.34.4.3 `repository`

```
src.application.services.TestReadService.repository
```

A documentação para essa classe foi gerada a partir do seguinte arquivo:

- [services.py](#)

10.35 Referência da Classe `src.infrastructure.cache.TTLCache`

Métodos Públicos

- `__init__` (self, int `ttl_seconds`)
- Any|None `get` (self, str key)
- None `set` (self, str key, Any value)

Atributos Públicos

- `ttl_seconds`

10.35.1 Descrição detalhada

@brief Cache LRU simplificado com expiração por tempo (TTL).

Thread-safe para uso single-thread (Streamlit). As entradas expiradas são removidas na próxima tentativa de leitura (lazy eviction).

10.35.2 Construtores e Destrutores

10.35.2.1 `__init__()`

```
src.infrastructure.cache.TTLCache.__init__ (
    self,
    int ttl_seconds )
```

@brief Inicializa o cache com TTL fixo.

@param ttl_seconds Tempo de vida padrão das entradas em segundos.

10.35.3 Documentação das funções

10.35.3.1 `get()`

```
Any | None src.infrastructure.cache.TTLCache.get (
    self,
    str key )
```

@brief Recupera um valor do cache, ou None se ausente/expirado.

@param key Chave da entrada.

@return Valor armazenado ou None.

10.35.3.2 `set()`

```
None src.infrastructure.cache.TTLCache.set (
    self,
    str key,
    Any value )
```

@brief Armazena um valor no cache com o TTL configurado.

@param key Chave da entrada.

@param value Valor a armazenar.

10.35.4 Atributos

10.35.4.1 `ttl_seconds`

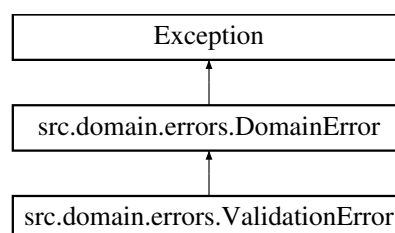
```
src.infrastructure.cache.TTLCache.ttl_seconds
```

A documentação para essa classe foi gerada a partir do seguinte arquivo:

- [cache.py](#)

10.36 Referência da Classe `src.domain.errors.ValidationError`

Diagrama de hierarquia da classe `src.domain.errors.ValidationError`:



10.36.1 Descrição detalhada

`@brief` Erro disparado quando uma regra de validação é violada.

A documentação para essa classe foi gerada a partir do seguinte arquivo:

- [errors.py](#)

Capítulo 11

Arquivos

11.1 Referência do Arquivo

ENGENHARIA_REVERSA_SQL_PROJETO_54.md

11.2 Referência do Arquivo

PERFORMANCE_REPORT_C01_TESTE_001.md

11.3 Referência do Arquivo

PERFORMANCE_REPORT_PROJETO54_HP_FINAL.md

11.4 Referência do Arquivo README.md

11.5 Referência do Arquivo dto.py

Componentes

- class [src.application.dto.LoadTestResponse](#)
- class [src.application.dto.ExportPdfRequest](#)

Namespaces

- namespace [src](#)
- namespace [src.application](#)
- namespace [src.application.dto](#)

11.6 Referência do Arquivo services.py

Componentes

- class [src.application.services.TestReadService](#)
- class [src.application.services.ObservationService](#)
- class [src.application.services.ExportService](#)
- class [src.application.services.DiagnosticsService](#)

Namespaces

- namespace [src](#)
- namespace [src.application](#)
- namespace [src.application.services](#)

11.7 Referência do Arquivo validators.py

Namespaces

- namespace [src](#)
- namespace [src.application](#)
- namespace [src.application.validators](#)

Funções

- None [src.application.validators.validate_cylinder](#) (int cyl_num)
- None [src.application.validators.validate_test_id](#) (str test_id)
- str|None [src.application.validators.validate_obs_text](#) (str|None obs_text)
- None [src.application.validators.validate_y_column_name](#) (str y_column_name)
- [ObservationCommand](#) [src.application.validators.validate_observation_command](#) ([ObservationCommand](#) cmd)
- None [src.application.validators.validate_observation_lookup](#) ([ObservationLookup](#) lookup)

11.8 Referência do Arquivo settings.py

Componentes

- class [src.config.settings.Settings](#)

Namespaces

- namespace [src](#)
- namespace [src.config](#)
- namespace [src.config.settings](#)

Funções

- bool [src.config.settings._as_bool](#) (str|None value, bool default=True)

Variáveis

- [src.config.settings.SETTINGS](#) = [Settings](#)()
Instância singleton das configurações do sistema.

11.9 Referência do Arquivo errors.py

Componentes

- class [src.domain.errors.DomainError](#)
- class [src.domain.errors.ValidationError](#)
- class [src.domain.errors.NotFoundError](#)
- class [src.domain.errors.AmbiguousMatchError](#)
- class [src.domain.errors.PersistenceError](#)
- class [src.domain.errors.InfrastructureError](#)

Namespaces

- namespace [src](#)
- namespace [src.domain](#)
- namespace [src.domain.errors](#)

11.10 Referência do Arquivo models.py

Componentes

- class [src.domain.models.TestContext](#)
- class [src.domain.models.ObservationCommand](#)
- class [src.domain.models.ObservationLookup](#)
- class [src.domain.models.ObservationResult](#)
- class [src.domain.models.ObservationRecord](#)
- class [src.domain.models.DynamicDataResult](#)
- class [src.domain.models.StaticDataResult](#)
- class [src.domain.models.DiagnosticsInfo](#)

Namespaces

- namespace [src](#)
- namespace [src.domain](#)
- namespace [src.domain.models](#)

11.11 Referência do Arquivo cache.py

Componentes

- class [src.infrastructure.cache._Entry](#)
- class [src.infrastructure.cache.TTLCache](#)

Namespaces

- namespace [src](#)
- namespace [src.infrastructure](#)
- namespace [src.infrastructure.cache](#)

11.12 Referência do Arquivo cache_redis.py

Componentes

- class [src.infrastructure.cache_redis._CacheEncoder](#)
- class [src.infrastructure.cache_redis.RedisCache](#)

Namespaces

- namespace [src](#)
- namespace [src.infrastructure](#)
- namespace [src.infrastructure.cache_redis](#)

Funções

- Any [src.infrastructure.cache_redis._cache_decoder](#) (dict dct)

Variáveis

- bool [src.infrastructure.cache_redis._HAS_REDIS](#) = True
- [src.infrastructure.cache_redis._redis](#) = None

11.13 Referência do Arquivo csv_exporter.py

Componentes

- class [src.infrastructure.exporters.csv_exporter.SemicolonCsvExporter](#)

Namespaces

- namespace [src](#)
- namespace [src.infrastructure](#)
- namespace [src.infrastructure.exporters](#)
- namespace [src.infrastructure.exporters.csv_exporter](#)

11.14 Referência do Arquivo pdf_exporter.py

Componentes

- class [src.infrastructure.exporters.pdf_exporter.TechnicalPdfExporter](#)

Namespaces

- namespace [src](#)
- namespace [src.infrastructure](#)
- namespace [src.infrastructure.exporters](#)
- namespace [src.infrastructure.exporters.pdf_exporter](#)

11.15 Referência do Arquivo __init__.py

Namespaces

- namespace [src](#)

11.16 Referência do Arquivo application/__init__.py

Namespaces

- namespace [src](#)
- namespace [src.application](#)

11.17 Referência do Arquivo config/__init__.py

Namespaces

- namespace [src](#)
- namespace [src.config](#)

11.18 Referência do Arquivo domain/__init__.py

Namespaces

- namespace [src](#)
- namespace [src.domain](#)

11.19 Referência do Arquivo infrastructure/__init__.py

Namespaces

- namespace [src](#)
- namespace [src.infrastructure](#)

11.20 Referência do Arquivo infrastructure/exporters/__init__.py

Namespaces

- namespace [src](#)
- namespace [src.infrastructure](#)
- namespace [src.infrastructure.exporters](#)

11.21 Referência do Arquivo infrastructure/sql/__init__.py

Namespaces

- namespace [src](#)
- namespace [src.infrastructure](#)
- namespace [src.infrastructure.sql](#)

11.22 Referência do Arquivo ui/__init__.py

Namespaces

- namespace [src](#)
- namespace [src.ui](#)

11.23 Referência do Arquivo connection.py

Componentes

- class [src.infrastructure.sql.connection.SqlServerConnection](#)

Namespaces

- namespace [src](#)
- namespace [src.infrastructure](#)
- namespace [src.infrastructure.sql](#)
- namespace [src.infrastructure.sql.connection](#)

Funções

- tuple[list[str], list[tuple[Any,...]]] [src.infrastructure.sql.connection._execute_on](#) (pyodbc.Connection conn, str query, Iterable[Any] params=())

11.24 Referência do Arquivo domain/repositories.py

Componentes

- class [src.domain.repositories.TestQueryRepository](#)
- class [src.domain.repositories.ObservationCommandRepository](#)
- class [src.domain.repositories.DiagnosticsRepository](#)
- class [src.domain.repositories.CsvExporter](#)
- class [src.domain.repositories.PdfExporter](#)

Namespaces

- namespace [src](#)
- namespace [src.domain](#)
- namespace [src.domain.repositories](#)

11.25 Referência do Arquivo infrastructure/sql/repositories.py

Componentes

- class [src.infrastructure.sql.repositories.SqlTestQueryRepository](#)
- class [src.infrastructure.sql.repositories.SqlObservationRepository](#)
- class [src.infrastructure.sql.repositories.SqlDiagnosticsRepository](#)

Namespaces

- namespace [src](#)
- namespace [src.infrastructure](#)
- namespace [src.infrastructure.sql](#)
- namespace [src.infrastructure.sql.repositories](#)

11.26 Referência do Arquivo safe_sql.py

Namespaces

- namespace [src](#)
- namespace [src.infrastructure](#)
- namespace [src.infrastructure.sql](#)
- namespace [src.infrastructure.sql.safe_sql](#)

Funções

- str [src.infrastructure.sql.safe_sql.escape_identifier](#) (str name)
- str [src.infrastructure.sql.safe_sql.escape_table](#) (str name)

Variáveis

- [src.infrastructure.sql.safe_sql._FORBIDDEN](#) = frozenset(['\x00'])

11.27 Referência do Arquivo schema.py

Namespaces

- namespace [src](#)
- namespace [src.infrastructure](#)
- namespace [src.infrastructure.sql](#)
- namespace [src.infrastructure.sql.schema](#)

Funções

- str [src.infrastructure.sql.schema.cylinder_id_column](#) (int cyl_num)
- str [src.infrastructure.sql.schema.dynamic_table_name](#) (int cyl_num)
- list[str] [src.infrastructure.sql.schema.static_table_candidates](#) (int cyl_num)

Variáveis

- dict [src.infrastructure.sql.schema.CANONICAL_OBSERVATION_COLUMNS](#)
Mapeamento de nomes canônicos para colunas de observação no banco.
- list [src.infrastructure.sql.schema.FORCE_COLUMN_CANDIDATES](#) = ["Forca", "For\u00e7a", "FORCA", "Carga", "Load"]
Candidatos a coluna de força (com e sem acento, português e inglês).
- dict [src.infrastructure.sql.schema.DYNAMIC_EXCLUDE_COLUMNS](#)
Colunas excluídas automaticamente das consultas dinâmicas.

- dict [src.infrastructure.sql.schema.DYNAMIC_DEFAULT_COLUMNS](#)

Conjunto padrão de colunas carregadas das tabelas dinâmicas.

11.28 Referência do Arquivo streamlit_app.py

Namespaces

- namespace [src](#)
- namespace [src.ui](#)
- namespace [src.ui.streamlit_app](#)

Funções

- tuple[[TestReadService](#), [ObservationService](#), [ExportService](#), [DiagnosticsService](#)] [src.ui.streamlit_app._build_services](#) ()
- go.Figure [src.ui.streamlit_app._plot_multiaxis](#) (pd.DataFrame df, list[str] y_cols)
- str [src.ui.streamlit_app._friendly_error](#) (Exception exc)
- None [src.ui.streamlit_app.main](#) ()

11.29 Referência do Arquivo theme.py

Namespaces

- namespace [src](#)
- namespace [src.ui](#)
- namespace [src.ui.theme](#)

Funções

- None [src.ui.theme.inject_theme](#) ()
- None [src.ui.theme.render_signature_flow](#) ()
- None [src.ui.theme.render_kpis](#) (*int total_rows, int y_selected, bool obs_ready)

